

Relatório Técnico da Plataforma USP Data Lake

Julho de 2026

Sumário

Capítulo 1 — Visão geral e delimitação do sistema	6
1.1 Objeto de análise	6
1.2 Organização funcional	6
1.3 Separação de responsabilidades	7
1.4 Limites das afirmações deste relatório	8
Capítulo 2 — Organização do repositório e arquitetura de software	9
2.1 Estrutura dos componentes executáveis	9
2.2 Camada de exposição HTTP	10
2.3 Arquitetura em camadas	10
2.4 Inicialização e ciclo de vida	10
2.5 Configuração por ambiente	11
2.6 Composição dos serviços	11
2.7 Estruturas paralelas observadas	12
Capítulo 3 — Interfaces de programação e contratos HTTP	13
3.1 Caracterização das APIs	13
3.2 Generator API	13
3.2.1 Verificação de integridade	13
3.2.2 Coleta de notícias	13
3.2.3 Aquisição e geração de PDF	14
3.3 Pipeline API	14
3.3.1 Rotas canônicas e aliases	15
3.3.2 Execução do processamento	15
3.4 RAG API	16
3.4.1 Rotas informativas	16
3.4.2 Contrato da pergunta	16
3.4.3 Fluxo da requisição	16
3.4.4 Tratamento de falhas	17
3.5 Métricas e registros da RAG API	17
Capítulo 4 — Pipeline de dados: Bronze, transformação e Gold	18
4.1 Delimitação das camadas	18
4.2 Camada Bronze e MinIO	18
4.3 Envio de artefatos Bronze	19
4.4 Modelo de documento Bronze	19
4.5 Leitura de JSON e PDF	19
4.6 Limpeza e normalização textual	20
4.7 Fragmentação do texto	20
4.8 Estrutura dos registros transformados	20

4.9 Transição para a camada Gold	21
4.10 Propriedades operacionais do pipeline	21
Capítulo 5 — Persistência Gold no PostgreSQL e no Qdrant	22
5.1 Distribuição da camada Gold	22
5.2 Configuração da conexão PostgreSQL	22
5.3 Esquema relacional	23
5.4 Upsert textual e transações	23
5.5 Normalização de URL	23
5.6 Resolução de colisões documentais	24
5.7 Configuração do Qdrant	24
5.8 Criação da coleção vetorial	24
5.9 Estrutura dos pontos vetoriais	25
5.10 Upsert vetorial em lotes	25
5.11 Integridade referencial entre os repositórios	25
Capítulo 6 — Embeddings e recuperação híbrida	26
6.1 Representação vetorial com BGE-M3	26
6.2 Codificação de textos	26
6.3 Configuração da recuperação	27
6.4 Tokenização lexical	27
6.5 Construção do índice BM25	27
6.6 Ciclo de vida do índice lexical	28
6.7 Busca vetorial	28
6.8 Reciprocal Rank Fusion ponderado	29
6.9 Modos de recuperação e fallback	29
6.10 Validação e deduplicação	29
6.11 Métricas do Retriever	30
Capítulo 7 — Evidence Check e controle de suficiência	31
7.1 Posição no fluxo da RAG	31
7.2 Configuração padrão	31
7.3 Extração dos termos temáticos	32
7.4 Sinais calculados por fonte	32
7.5 Classes de evidência	32
7.6 Evidência parcial forte	33
7.7 Critério de aceitação por múltiplas fontes	33
7.8 Critério de fonte única forte	34
7.9 Regras de recusa	34
7.10 Fontes para contexto e para resposta pública	34
7.11 Resultado e métricas	35
7.12 Limite semântico do mecanismo	35
Capítulo 8 — Preparação de contexto e geração local com Ollama	36
8.1 Responsabilidade do serviço de linguagem	36
8.2 Configuração do Ollama	36
8.3 Reutilização do cliente HTTP	37
8.4 Filtragem de chunks	37
8.5 Agrupamento por documento	37
8.6 Construção e truncamento do contexto	37
8.7 Prompt de sistema	38
8.8 Requisição à API de chat	38

8.9 Interpretação da resposta	39
8.10 Taxonomia de falhas	39
8.11 Dependência exclusivamente local no código analisado	40
Capítulo 9 — Containerização e infraestrutura de execução	41
9.1 Imagem comum das APIs	41
9.2 Dependências Python	41
9.3 Composição principal	41
9.4 MinIO	42
9.5 PostgreSQL	42
9.6 Qdrant	42
9.7 Execução das APIs	43
9.8 Dependências declaradas	43
9.9 Health checks das APIs	43
9.10 Cache local do modelo de embeddings	44
9.11 Comunicação com o Ollama no hospedeiro	44
9.12 Volumes e persistência	44
9.13 Arquivo Compose alternativo	45
9.14 Variáveis exemplificadas no repositório	45
9.15 Delimitação de segurança da configuração	45
Capítulo 10 — Testes automatizados e avaliação quantitativa	46
10.1 Organização da verificação	46
10.2 Testes de contratos HTTP	46
10.3 Testes do fluxo PDF	47
10.4 Testes de identidade do corpus	47
10.5 Testes do Retriever híbrido	47
10.6 Testes do Evidence Check	48
10.7 Testes de contexto e Ollama	48
10.8 Testes do ciclo de vida da RAG API	48
10.9 Estrutura do benchmark	49
10.10 Validação do ground truth	49
10.11 Execução do benchmark	50
10.12 Métricas de classificação e recuperação	50
10.13 Latência e avaliação humana	50
10.14 Proteção contra conclusões prematuras	51
10.15 Alcance da evidência de testes	51
Capítulo 11 — Scripts operacionais e correspondência do README	52
11.1 Scripts auxiliares	52
11.2 Construção do manifesto do corpus	52
11.3 Validação das URLs existentes	53
11.4 Validação curta de demonstração	53
11.5 Validação ampliada do corpus	53
11.6 Arquitetura descrita no README	54
11.7 Serviços, portas e endpoints no README	54
11.8 Parâmetros da RAG documentados	54
11.9 Delimitação do corpus no README	54
11.10 Contagem de testes	55
11.11 Instalação e execução documentadas	55
11.12 Limites de correspondência documental	55

Capítulo 12 — Síntese técnica, limitações e conclusão	56
12.1 Síntese arquitetural	56
12.2 Propriedades de modularidade	56
12.3 Propriedades de rastreabilidade	57
12.4 Propriedades de controle factual	57
12.5 Persistência e idempotência	57
12.6 Limitações da camada de processamento	57
12.7 Limitações da recuperação	58
12.8 Limitações do Evidence Check	58
12.9 Limitações da geração	58
12.10 Limitações dos health checks	59
12.11 Limitações de configuração e implantação	59
12.12 Limitações de validação	59
12.13 Conclusão	60
Apêndice A — Diagramas do sistema	61

Índice de Figuras

1	Arquitetura geral da plataforma	62
2	Pipeline de dados Bronze, Silver e Gold	63
3	Fluxo interno da RAG	64
4	Fluxo das APIs	65
5	Persistência e identidade dos dados	66
6	Fluxo conversacional do chatbot	67

Capítulo 1 — Visão geral e delimitação do sistema

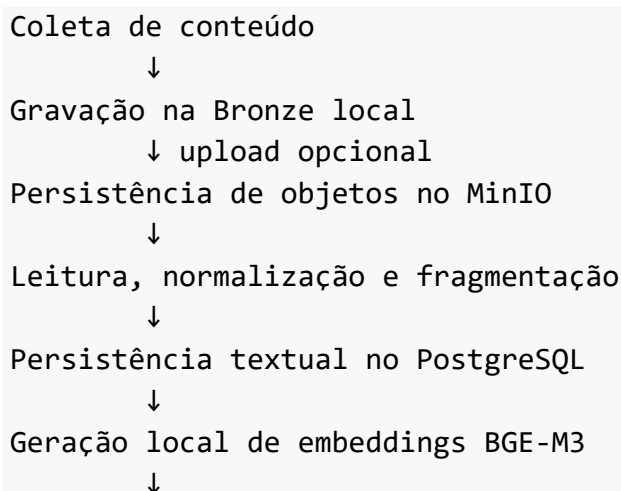
1.1 Objeto de análise

Este relatório descreve a implementação de uma plataforma de coleta, transformação, persistência e recuperação de conteúdo digital. A delimitação adotada decorre diretamente dos módulos do pacote `src`: a coleta é implementada em `src/scrapper.py`; o armazenamento de objetos da camada de entrada, em `src/minio_client.py` e `src/upload_bronze.py`; a transformação e a fragmentação textual, em `src/transform.py` e `src/chunking.py`; a persistência estruturada e vetorial, em `src/postgres_loader.py` e `src/qdrant_loader.py`; e a recuperação com geração de resposta, em `src/retriever.py`, `src/llm_service.py` e `src/api/rag_api.py`.

O sistema disponibiliza três aplicações FastAPI independentes. `src/api/generator_api.py` define a API de geração; `src/api/pipeline_api.py` define a API de processamento; e `src/api/rag_api.py` define a API de consulta. O arquivo `docker-compose.yml` associa essas aplicações, respectivamente, às portas externas 8001, 8002 e 8003 e as inicia por meio do Uvicorn.

1.2 Organização funcional

A organização funcional observada no código pode ser sintetizada no seguinte fluxo:



```
Persistência vetorial no Qdrant
↓
Recuperação lexical e vetorial
↓
Fusão de rankings e avaliação de evidência
↓
Geração de resposta pelo Ollama
```

A API de geração utiliza funções de `src/scrapper.py` para coletar e salvar artigos, podendo enviar os arquivos resultantes ao MinIO por meio de `src/upload_bronze.py`. A API de processamento chama `run_transform`, definido em `src/transform.py`, e pode encaminhar os registros resultantes para `insert_chunks`, em `src/postgres_loader.py`, e para `upsert_embeddings`, em `src/qdrant_loader.py`. Antes do carregamento vetorial, `src/api/pipeline_api.py` instancia `BGEM3Embedder` e executa `embed_records`.

Na consulta, `src/api/rag_api.py` obtém uma instância de `Retriever`, executa a recuperação de candidatos e submete o resultado a `evaluate_evidence`. Quando a evidência é considerada suficiente, a API limita e organiza o contexto com `prepare_context` e solicita a resposta a `generate_answer`, ambas definidas em `src/llm_service.py`. Quando a evidência é insuficiente, o fluxo retorna uma resposta controlada sem executar a geração pelo modelo de linguagem.

1.3 Separação de responsabilidades

O código estabelece separação entre quatro responsabilidades principais:

1. **Ingestão:** `src/scrapper.py`, `src/upload_bronze.py` e `src/minio_client.py` tratam da obtenção e do armazenamento inicial dos objetos.
2. **Processamento:** `src/transform.py` e `src/chunking.py` convertem os documentos de entrada em registros textuais fragmentados.
3. **Persistência:** `src/postgres_loader.py` mantém texto e metadados, enquanto `src/qdrant_loader.py` mantém as representações vetoriais.
4. **Consulta:** `src/retriever.py` combina mecanismos de recuperação; `src/api/rag_api.py` controla a requisição; e `src/llm_service.py` prepara o contexto e se comunica com o Ollama.

Essa separação também aparece no `docker-compose.yml`. MinIO, PostgreSQL e Qdrant são declarados como serviços distintos, e cada API possui comando, porta, variáveis de ambiente e verificação de integridade próprios. O serviço da RAG não declara um contêiner Ollama no Compose: a variável `OLLAMA_BASE_URL` utiliza, por padrão, `http://host.docker.internal:11434`, indicando que a aplicação acessa uma instância disponível no hospedeiro.

A dependência declarada entre Generator API e MinIO no Compose não torna o upload obrigatório no contrato HTTP. Os endpoints da Generator gravam primeiro na Bronze local; o envio ao MinIO ocorre somente quando o parâmetro `upload_minio` é verdadeiro.

1.4 Limites das afirmações deste relatório

As descrições apresentadas baseiam-se exclusivamente em estruturas efetivamente observáveis no repositório: definições de funções e classes, modelos Pydantic, rotas FastAPI, instruções de persistência, algoritmos de recuperação, testes automatizados e configurações de contêineres. A existência de uma implementação ou de um teste demonstra a intenção e o comportamento codificado, mas não comprova, isoladamente, que serviços externos estejam ativos ou que determinada quantidade de dados esteja armazenada em uma execução concreta. Por esse motivo, valores operacionais dependentes do estado de PostgreSQL, Qdrant, MinIO ou Ollama não são assumidos neste capítulo.

Capítulo 2 — Organização do repositório e arquitetura de software

2.1 Estrutura dos componentes executáveis

O pacote `src` concentra os componentes utilizados pelos comandos de execução declarados no `docker-compose.yml`. Sua organização distribui as responsabilidades da aplicação da seguinte forma:

```
src/
├── api/
│   ├── generator_api.py
│   ├── pipeline_api.py
│   └── rag_api.py
├── scraper.py
├── upload_bronze.py
├── minio_client.py
├── transform.py
├── chunking.py
├── postgres_loader.py
├── embedding.py
├── qdrant_loader.py
├── retriever.py
└── llm_service.py
```

Os arquivos `src/__init__.py` e `src/api/__init__.py` caracterizam `src` e `src.api` como pacotes Python. Essa estrutura permite que o Uvicorn importe as aplicações pelos caminhos `src.api.generator_api:app`, `src.api.pipeline_api:app` e `src.api.rag_api:app`, exatamente como especificado nos comandos dos serviços do Compose.

Os módulos posicionados diretamente em `src` não constituem aplicações HTTP independentes. Eles fornecem operações utilizadas pelas APIs ou por interfaces de linha de comando. `src/scraper.py`, `src/upload_bronze.py`, `src/transform.py`, `src/postgres_loader.py`, `src/embedding.py` e `src/qdrant_loader.py` possuem função `main`, permitindo execução direta para atividades específicas. Em paralelo, as APIs importam suas funções e classes para compor fluxos acionados por requisições HTTP.

2.2 Camada de exposição HTTP

Cada módulo oficial em `src/api` instancia seu próprio objeto FastAPI. Não há, no `docker-compose.yml`, um processo único agregando as três aplicações. Cada serviço executa uma instância separada do Uvicorn, embora todos sejam construídos com o mesmo Dockerfile da raiz e compartilhem o diretório `/workspace` como diretório de trabalho.

A Generator API importa operações de coleta de `src.scrapers` e a rotina de envio ao armazenamento de objetos de `src.upload_bronze`. A Pipeline API importa o transformador, o carregador PostgreSQL, o gerador de embeddings e o carregador Qdrant. A RAG API importa o Retriever, a avaliação de evidência e o serviço responsável pela preparação de contexto e comunicação com o Ollama. Essas relações de importação estabelecem dependências direcionadas das APIs para os módulos de domínio e infraestrutura.

2.3 Arquitetura em camadas

Com base nessas dependências, a implementação pode ser organizada em cinco camadas lógicas:

1. **Exposição:** constituída pelos módulos FastAPI em `src/api`, responsáveis por validar entradas, declarar contratos HTTP e converter resultados ou falhas em respostas.
2. **Aquisição:** constituída por `src/scrapers.py`, que obtém e extrai conteúdo, e por `src/upload_bronze.py`, que encaminha artefatos ao armazenamento Bronze.
3. **Transformação:** constituída por `src/transform.py` e `src/chunking.py`, que interpretam objetos JSON ou PDF, normalizam o conteúdo e produzem chunks.
4. **Persistência e representação:** constituída por `src/postgres_loader.py`, `src/embedding.py`, `src/qdrant_loader.py` e `src/minio_client.py`.
5. **Recuperação e geração:** constituída por `src/retriever.py` e `src/llm_service.py`, sob coordenação de `src/api/rag_api.py`.

Essa classificação é lógica, pois o código não define pacotes físicos separados para cada camada. Ainda assim, os imports preservam a distinção operacional: por exemplo, `src/retriever.py` obtém conexões por meio de `get_postgres_connection`, obtém o cliente vetorial por meio de `get_qdrant_client` e utiliza `BGEM3Embedder` para representar a consulta.

2.4 Inicialização e ciclo de vida

A Generator API e a Pipeline API criam seus objetos FastAPI sem um gerenciador de ciclo de vida próprio. Recursos necessários às operações são obtidos durante a execução dos respectivos endpoints. Na Pipeline API, a função `_run_pipeline` instancia `BGEM3Embedder` quando o carregamento no Qdrant é solicitado, após a transformação e a persistência textual.

A RAG API adota estratégia distinta. A função assíncrona `lifespan`, registrada na construção do objeto FastAPI, cria uma instância de `Retriever`, chama seu método `initialize` e a armazena em `app.state.retriever`. Um lock global protege a seção de inicialização, e

o código verifica se uma instância já está armazenada antes de criar outra. No encerramento, o método `close` do `Retriever` é chamado e o cliente HTTP compartilhado do serviço de linguagem é fechado por `close_http_client`.

O `Retriever`, por sua vez, mantém referências ao `embedder`, ao cliente `Qdrant`, ao nome da coleção e ao índice `BM25`. As propriedades `embedder`, `qdrant` e `collection` realizam inicialização tardia quando os objetos ainda não foram atribuídos. A codificação da pergunta é protegida por `_embedding_lock`, enquanto a criação e o acesso ao índice lexical utilizam `_lexical_lock`. Essa implementação separa o carregamento dos recursos do momento de importação dos módulos.

2.5 Configuração por ambiente

Os componentes obtêm parâmetros de execução por variáveis de ambiente. Entre as configurações observáveis no código estão as credenciais e o endpoint do MinIO, a conexão do PostgreSQL, o endereço e a coleção do Qdrant, o nome do modelo de embeddings, os parâmetros da recuperação híbrida, os limiares do Evidence Check e as opções de comunicação com o Ollama.

O `docker-compose.yml` fornece essas variáveis aos serviços correspondentes. Para `Pipeline API` e `RAG API`, também define `EMBEDDING_MODEL` como `BAAI/bge-m3` e configura diretórios de cache do Hugging Face. As variáveis `HF_HUB_OFFLINE` e `TRANSFORMERS_OFFLINE` recebem o valor `1`, e o cache do hospedeiro é montado em `/models/huggingface` como somente leitura. No construtor de `BGEM3Embedder`, a configuração `HF_HUB_OFFLINE` determina o argumento `local_files_only` usado na criação do `SentenceTransformer`.

2.6 Composição dos serviços

O `docker-compose.yml` declara seis serviços: `minio`, `postgres`, `qdrant`, `generator-api`, `pipeline-api` e `rag-api`. O acesso ao Ollama é externo à composição, pois ele não é declarado como serviço do arquivo. Portanto, há seis contêineres definidos pelo Compose e uma dependência de geração de linguagem acessada por URL.

As relações `depends_on` expressam a ordem de dependência declarada: a `Generator API` depende do MinIO; a `Pipeline API` depende de MinIO, PostgreSQL e Qdrant; e a `RAG API` depende de PostgreSQL e Qdrant. MinIO, PostgreSQL e Qdrant utilizam volumes nomeados próprios. As três APIs montam o repositório em `/workspace`, e `Pipeline API` e `RAG API` também montam o cache do modelo de embeddings.

O repositório contém ainda `docker/docker-compose.yml`. Esse arquivo aponta seu contexto de construção e volume de trabalho para o diretório pai e inicia os mesmos módulos oficiais em `src/api`. Entretanto, ele não contém todas as variáveis de cache, modo offline e parametrização da RAG presentes no Compose da raiz. Assim, os dois arquivos descrevem os mesmos papéis de serviço, mas não são equivalentes em todas as configurações operacionais.

2.7 Estruturas paralelas observadas

Além dos módulos utilizados pelos comandos do Compose, o repositório contém os diretórios `generator-api` e `pipeline-api`, bem como `src/api/main.py` e `Dockerfile-api`. Os comandos dos dois arquivos Compose examinados não utilizam esses pontos de entrada: ambos referenciam o `Dockerfile` da raiz e as aplicações `src.api.generator_api`, `src.api.pipeline_api` e `src.api.rag_api`.

Consequentemente, a arquitetura executada pelas configurações de contêiner analisadas é a implementação localizada em `src/api`. A presença das estruturas paralelas não é suficiente para determinar sua origem histórica, mas sua ausência nos comandos de inicialização permite afirmar que elas não constituem os pontos de entrada definidos nos arquivos Compose atuais.

Capítulo 3 — Interfaces de programação e contratos HTTP

3.1 Caracterização das APIs

As interfaces HTTP são implementadas com FastAPI e executadas pelo Uvicorn. Cada aplicação informa título, descrição e versão `1.0.0` na construção do objeto FastAPI, o que permite ao próprio framework produzir o esquema OpenAPI correspondente. As funções de rota utilizam anotações de tipo, parâmetros Query ou modelos Pydantic para declarar as restrições de entrada.

O `docker-compose.yml` publica a porta interna 8000 de cada aplicação em uma porta distinta do hospedeiro: 8001 para Generator API, 8002 para Pipeline API e 8003 para RAG API.

3.2 Generator API

A Generator API é definida em `src/api/generator_api.py`. Sua finalidade codificada é coletar conteúdo e salvá-lo no diretório `bronze/raw`, com envio opcional ao MinIO.

3.2.1 Verificação de integridade

O endpoint `GET /health` não recebe parâmetros e retorna diretamente:

```
{
  "status": "ok",
  "service": "generator-api"
}
```

Essa função não chama o scraper nem instancia um cliente MinIO.

3.2.2 Coleta de notícias

O endpoint `GET /site` aceita três parâmetros de consulta:

Parâmetro	Tipo	Padrão	Restrições declaradas
<code>limit</code>	inteiro	10	mínimo 1 e máximo 500
<code>max_pages</code>	inteiro ou nulo	nulo	mínimo 1 e máximo 100

Parâmetro	Tipo	Padrão	Restrições declaradas
upload_minio	booleano	false	sem restrição adicional

A rota chama `run`, importada de `src.scrapar`, fornecendo o limite, a raiz do projeto e o número máximo de páginas. Se `upload_minio` for verdadeiro, chama `upload_bronze_files` para enviar os arquivos do diretório Bronze.

Em caso de sucesso, `_response` padroniza o corpo com `status`, `service`, `message` e `data`. O objeto `data` informa a quantidade de artigos retornados pelo scraper, o diretório Bronze e, quando solicitado, o total de uploads concluídos e falhos. Exceções não tratadas internamente são convertidas em `HTTPException` de código 500, com a representação textual da exceção no campo `detail`.

3.2.3 Aquisição e geração de PDF

O endpoint GET `/pdf` aceita:

Parâmetro	Tipo	Obrigatoriedade	Restrições declaradas
url	string	obrigatório	comprimento mínimo 1; esquema HTTP ou HTTPS e host válido
filename	string ou nulo	opcional	comprimento entre 1 e 255
upload_minio	booleano	opcional	padrão false

A rota realiza uma requisição HTTP com `requests.get`, usando `USER_AGENT` e `REQUEST_TIMEOUT` importados de `src.scrapar`. Se a resposta começar com a assinatura `%PDF-`, o conteúdo é preservado. Se a resposta for HTML, ou o host pertencer a `journal.usp.br`, o código extrai título e conteúdo e gera um PDF textual por `_build_text_pdf`. Outros tipos de conteúdo produzem resposta HTTP 400.

O nome final passa por `_safe_pdf_filename`, que substitui caracteres fora do conjunto alfanumérico, ponto, sublinhado e hífen, garante uma alternativa não vazia e acrescenta a extensão `.pdf` quando necessário. O arquivo é gravado em `bronze/raw`; o envio ao MinIO ocorre somente quando solicitado. A resposta padronizada inclui caminho, nome, URL de origem, tamanho em bytes e informações do objeto MinIO.

3.3 Pipeline API

A Pipeline API é definida em `src/api/pipeline_api.py`. Seu contrato de entrada é centralizado no modelo `Pydantic ProcessRequest`:

Campo	Tipo	Padrão	Validação
source	string	minio	expressão regular limitada a minio ou local
limit	inteiro ou nulo	nulo	mínimo 1 quando informado
load_postgres	booleano	true	—
load_qdrant	booleano	true	—

O endpoint GET /health retorna status=ok e service=pipeline-api, sem executar transformação ou conexão explícita com os repositórios de dados.

3.3.1 Rotas canônicas e aliases

As rotas canônicas são:

- POST /processar-site, que chama _run_pipeline com a extensão json e o prefixo padrão raw/;
- POST /processar-pdf, que chama _run_pipeline com a extensão pdf e o prefixo retornado por get_pdf_prefix.

As rotas POST /site e POST /pdf encaminham a requisição para as funções canônicas correspondentes. Ambas foram declaradas com deprecated=True, informação que passa a integrar o esquema OpenAPI.

3.3.2 Execução do processamento

_run_pipeline executa inicialmente run_transform. Quando nenhum registro é produzido, retorna apenas a mensagem Nenhum documento novo para processar.. Havendo registros e estando load_postgres habilitado, a função resolve colisões de identidade por resolve_document_id_collisions e persiste os chunks por insert_chunks.

Se load_qdrant estiver habilitado, a função instancia BGEM3Embedder, codifica o campo texto de todos os registros e chama upsert_embeddings. A resposta contabiliza documentos, chunks, registros PostgreSQL, vetores Qdrant, caracteres, objetos de origem e documentos remapeados. O código distingue a quantidade total processada da quantidade considerada nova por meio do campo _inserted retornado pelo carregador PostgreSQL. Quando a persistência PostgreSQL está habilitada e nenhum registro é novo, a mensagem caracteriza o processamento como idempotente.

Qualquer exceção propagada pelo fluxo é convertida em resposta HTTP 500, e seu texto é utilizado como detail.

O modelo ProcessRequest não contém validação cruzada entre load_postgres e load_qdrant. Por isso, load_postgres=false com load_qdrant=true é aceito pelo contrato. Em registros recém-transformados, porém, essa combinação normalmente falha:

`_point_id`, no carregador Qdrant, exige `postgres_id` ou `id`, e essa identidade é acrescentada por `insert_chunks`. O fluxo Gold completo deve, portanto, carregar o PostgreSQL antes do Qdrant.

3.4 RAG API

A RAG API é definida em `src/api/rag_api.py`. Ela expõe informações do serviço e uma operação de pergunta baseada em recuperação.

3.4.1 Rotas informativas

GET / retorna `status`, `service` e a mensagem USP Data Lake - RAG API. GET /health retorna `status=ok` e `service=rag-api`. Nenhuma dessas funções acessa o Retriever ou o Ollama.

3.4.2 Contrato da pergunta

POST /pergunta recebe o modelo `PerguntaRequest`:

Campo	Tipo	Padrão	Validação
<code>pergunta</code>	<code>string</code>	obrigatório	comprimento mínimo 3
<code>top_k</code>	<code>inteiro</code>	5	mínimo 1 e máximo 20

A resposta é validada pelo modelo `PerguntaResponse`, composto por `status`, `service`, `pergunta`, `resposta`, `fontes`, `total_fontes`, `evidence_sufficient`, `ollama_skipped` e `metrics`.

Cada fonte segue `FonteResponse` e pode conter `id`, `titulo`, `url`, `texto`, `score`, `document_id`, `chunk_id`, `postgres_id`, `source_type` e `source_object`. A função `_fonte_from_chunk` converte o registro interno para esse contrato. `_derive_source_type` classifica a origem como `json`, `pdf` ou `unknown` a partir do objeto de origem, de campos alternativos ou da extensão da URL.

3.4.3 Fluxo da requisição

A operação obtém o Retriever armazenado em `app.state`. Sua ausência produz HTTP 503. Em seguida, chama `search_with_metrics` com a pergunta e `top_k`. O resultado é avaliado por `evaluate_evidence`.

Quando a evidência é insuficiente, a API não prepara uma chamada ao Ollama. Ela retorna a constante `NO_RESULTS_MESSAGE`, marca `evidence_sufficient` como falso e `ollama_skipped` como verdadeiro. As fontes públicas são as selecionadas pela avaliação de evidência, e as métricas de contexto e Ollama recebem zero.

Quando a evidência é suficiente, `prepare_context` produz o contexto e os chunks selecionados. Havendo chunks, `generate_answer` realiza a geração; na ausência deles, a própria API

utiliza a mensagem de insuficiência. A resposta bem-sucedida marca `evidence_sufficient` como verdadeiro e `ollama_skipped` como falso e retorna como fontes os chunks efetivamente selecionados para o contexto.

3.4.4 Tratamento de falhas

O endpoint diferencia classes de falha:

- erros de tipo ou valor na recuperação resultam em HTTP 400;
- indisponibilidade ou timeout dos serviços de recuperação resultam em HTTP 503;
- indisponibilidade do Ollama ou modelo ausente resultam em HTTP 503;
- resposta inválida do Ollama resulta em HTTP 502;
- falhas de configuração do serviço de linguagem ou erros inesperados resultam em HTTP 500.

Nos erros relacionados ao Retriever e ao Ollama, as mensagens HTTP são controladas e não incluem o texto da exceção original. O operador `raise ... from error` preserva o encadeamento interno da exceção, mas o corpo enviado ao cliente utiliza a mensagem definida em cada `HTTPException`.

3.5 Métricas e registros da RAG API

A RAG API registra métricas por meio do logger `uvicorn.error`. `_log_metrics` serializa o dicionário em JSON. `_log_evidence_diagnostics` registra a decisão geral e, em nível de depuração, os sinais de cada candidato, incluindo scores, rankings, termos cobertos e metadados. O texto integral do documento não é incluído nesse registro de diagnóstico.

Somente um subconjunto das métricas internas é exposto na resposta pública. `_public_metrics` permite tempos de embedding, Qdrant, PostgreSQL, Retriever, Evidence Check, preparação do contexto, Ollama e tempo total, além do tamanho do contexto, número de fontes, candidatos deduplicados, decisão de evidência e motivo da recusa.

Capítulo 4 — Pipeline de dados: Bronze, transformação e Gold

4.1 Delimitação das camadas

O pipeline implementa uma sequência entre objetos de entrada, registros textuais transformados e duas formas de persistência para consulta. A terminologia presente nos módulos e nas descrições das APIs identifica os objetos JSON e PDF como Bronze, os chunks transformados como Silver e os registros persistidos no PostgreSQL e no Qdrant como Gold.

No fluxo acionado pela Pipeline API, a representação Silver existe em memória como uma lista de dicionários retornada por `run_transform`. A gravação dessa lista em JSON Lines está disponível na interface de linha de comando de `src/transform.py`, mas `_run_pipeline`, em `src/api/pipeline_api.py`, encaminha os registros diretamente aos carregadores Gold. Assim, o código da API não exige um arquivo Silver intermediário para concluir a persistência.

4.2 Camada Bronze e MinIO

`src/minio_client.py` encapsula a configuração do cliente MinIO. O endpoint, as credenciais, o uso de TLS e o bucket são obtidos, respectivamente, das variáveis `MINIO_ENDPOINT`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `MINIO_SECURE` e `MINIO_BUCKET`. Na ausência dessas variáveis, o código define `localhost:9000`, `minioadmin`, `minioadmin123`, conexão sem TLS e bucket `bronze`.

A Bronze local em `bronze/raw` é o primeiro destino dos artefatos produzidos pela Generator API. O MinIO constitui uma persistência opcional dessa camada: `upload_bronze_file` ou `upload_bronze_files` somente são chamados pelos endpoints quando `upload_minio=true`.

Os objetos são organizados por prefixo. `get_json_prefix` utiliza `raw/` como padrão, enquanto `get_pdf_prefix` utiliza `raw/pdf/`. A função `_normalized_prefix` remove barras excedentes e garante uma única barra ao final de um prefixo não vazio.

`ensure_bucket` consulta a existência do bucket e o cria quando ausente. `upload_file` determina o tipo de conteúdo pelo módulo `mimetypes` e utiliza `fput_object` para enviar um arquivo local. Falhas `S3Error` são convertidas em `RuntimeError` com identificação do caminho local e do objeto de destino.

4.3 Envio de artefatos Bronze

`src/upload_bronze.py` disponibiliza operações para um arquivo ou para um conjunto de arquivos. `upload_bronze_file` rejeita caminhos inexistentes, arquivos vazios e extensões diferentes de `.json` e `.pdf`. Após o envio, consulta o objeto com `stat_object` e retorna bucket, chave, tamanho e tipo de conteúdo.

`upload_bronze_files` enumera arquivos JSON e PDF presentes em `bronze/raw`, ordenando separadamente cada conjunto. A função cria ou reutiliza o bucket e envia cada arquivo ao prefixo correspondente. O retorno contém as quantidades de uploads concluídos e falhos. A ausência de arquivos produz `FileNotFoundError`; uma falha de conexão representada por `MaxRetryError` é convertida em `ConnectionError`.

4.4 Modelo de documento Bronze

O transformador define a estrutura imutável `BronzeDocument`, composta por `object_name`, `documento_id` e `payload`. `object_name` identifica o objeto de origem, `documento_id` identifica logicamente o documento e `payload` contém os campos recebidos ou extraídos.

Para um JSON cujo conteúdo seja um objeto, `_decode_json_object` utiliza como identificador o nome do arquivo sem extensão. Quando o JSON contém uma lista, cada elemento que seja um dicionário origina um documento; o identificador combina o nome base e um índice de seis algarismos iniciado em 1. Valores JSON que não sejam objeto nem lista de objetos não produzem documentos.

Para PDFs, `_decode_pdf_bytes` cria um único documento. O identificador recebe o prefixo `pdf_` e o nome do objeto sem extensão. O título é lido do metadado `/Title` ou, quando ausente, derivado do nome do arquivo. Se o metadado `/Subject` começar com `Source-URL:`, o restante é utilizado como URL de origem. Autor, data e categoria são inicializados como strings vazias.

4.5 Leitura de JSON e PDF

O pipeline pode ler objetos do MinIO ou arquivos locais. `iter_bronze_documents` seleciona `iter_minio_documents` quando `source` é `minio` e `iter_local_documents` quando é `local`; outros valores produzem `ValueError`.

Na leitura MinIO, o código verifica inicialmente a existência do bucket, enumera objetos recursivamente sob o prefixo solicitado e filtra suas extensões. Cada resposta de `get_object` é lida integralmente e, em um bloco `finally`, fechada e liberada por `close` e `release_conn`.

Na leitura local, os arquivos do diretório Bronze são ordenados e filtrados por extensão. O limite, quando informado, é aplicado à quantidade de documentos produzidos, e não diretamente à quantidade de chunks.

O JSON é decodificado como UTF-8 e interpretado com `json.loads`. O PDF precisa ser não vazio e começar com `%PDF-`. `PyPDF2.PdfReader` extrai o texto de todas as páginas,

concatenando-as com duas quebras de linha. Um arquivo sem texto extraível ou com formato inválido provoca `RuntimeError`.

4.6 Limpeza e normalização textual

A transformação de HTML em texto utiliza BeautifulSoup e markdownify. `limpar_html_para_markdown` remove comentários e elementos selecionados como ruído, incluindo scripts, estilos, formulários, navegação, cabeçalhos, rodapés, conteúdo lateral, widgets de compartilhamento e áreas de newsletter. Em seguida, converte o HTML para Markdown com cabeçalhos ATX e marcadores por hífen.

`_normalize_markdown` executa as seguintes operações codificadas:

- resolve entidades HTML;
- tenta reparar sequências indicativas de UTF-8 interpretado como Latin-1 ou Windows-1252;
- uniformiza quebras de linha e espaços não separáveis;
- remove imagens Markdown;
- transforma determinados links internos ou JavaScript em texto simples;
- reduz espaços e linhas em branco consecutivas;
- elimina linhas que correspondam aos padrões de compartilhamento, publicidade e referências como “leia também”.

`montar_texto_limpo` combina o título normalizado, precedido por #, e o conteúdo convertido. Campos auxiliares como autor, data, categoria e URL são normalizados por `_clean_plain_value` antes de integrar cada registro.

4.7 Fragmentação do texto

O chunking está implementado em `src/chunking.py`. Os valores padrão são 1.200 caracteres por chunk e sobreposição de 200 caracteres. `criar_chunks` valida que o tamanho seja positivo, que a sobreposição não seja negativa e que seja menor que o tamanho do chunk.

Após remover espaços nas extremidades do texto, a função percorre o conteúdo com passo igual a `chunk_size - overlap`. Cada fragmento também tem seus espaços externos removidos. O algoritmo opera por posição de caractere; não há, nessa função, segmentação por sentença, parágrafo ou token. Um texto vazio resulta em lista vazia.

4.8 Estrutura dos registros transformados

`transformar_documento` enumera os chunks a partir de 1. Para cada fragmento, produz um dicionário com:

- `documento_id` e `chunk_id`;
- `texto`;
- `titulo`, `autor`, `data_publicacao`, `categoria` e `url`;

- `source_object`;
- `chunk_size` e `overlap`.

`run_transform` seleciona a origem, percorre os documentos e agrega os registros resultantes. Ao final, imprime a quantidade de documentos e chunks e retorna a lista completa.

4.9 Transição para a camada Gold

Na Pipeline API, a persistência PostgreSQL antecede a persistência Qdrant. Quando habilitada, a etapa textual executa `resolve_document_id_collisions` e `insert_chunks`. O retorno de `insert_chunks` adiciona aos registros a identidade produzida pelo PostgreSQL; essa mesma lista é então utilizada na geração e no carregamento dos vetores.

A etapa vetorial cria `BGEM3Embedder`, codifica o texto de cada registro e chama `upsert_embeddings`. O carregador Qdrant exige que cada registro possua `postgres_id` ou `id`; portanto, a associação implementada para o fluxo completo depende da identidade obtida na persistência PostgreSQL.

A interface de linha de comando de `src/transform.py` aplica a mesma precedência quando o carregamento Qdrant é solicitado: a expressão que calcula `load_postgres` o habilita também para `--load-qdrant`, garantindo que os registros recebam IDs antes do upsert vetorial. Ela ainda permite salvar os chunks em `data/chunks.jsonl`, omitir esse arquivo com `--no-output` e configurar tamanho, sobreposição, modelo, dispositivo e lote de embeddings.

4.10 Propriedades operacionais do pipeline

O código apresenta três propriedades verificáveis. Primeiro, o mesmo transformador atende JSON e PDF, mas preserva o objeto e o tipo de entrada por meio de `source_object` e dos prefixos. Segundo, o limite é aplicado durante a produção de documentos, antes do chunking. Terceiro, a Pipeline API permite desligar individualmente os carregamentos PostgreSQL e Qdrant, embora o carregador Qdrant, por sua própria validação, necessite que os registros fornecidos já tenham uma identidade PostgreSQL compatível.

Capítulo 5 — Persistência Gold no PostgreSQL e no Qdrant

5.1 Distribuição da camada Gold

A camada Gold é distribuída entre dois sistemas de persistência com funções distintas. O PostgreSQL armazena o texto integral dos chunks e seus metadados; o Qdrant armazena os vetores e um subconjunto de metadados necessário à identificação dos pontos. A associação entre as duas representações é estabelecida pelo identificador numérico produzido pelo PostgreSQL, utilizado também como identificador do ponto Qdrant.

Essa sequência é explícita em `src/api/pipeline_api.py`: `insert_chunks` retorna registros enriquecidos com `id` e `postgres_id`; posteriormente, esses registros são fornecidos a `upsert_embeddings`. O próprio carregador vetorial rejeita registros sem uma dessas identidades.

5.2 Configuração da conexão PostgreSQL

`src/postgres_loader.py` carrega variáveis do arquivo `.env` localizado na raiz do projeto. `get_postgres_connection` utiliza `POSTGRES_DSN` quando essa variável está definida. Caso contrário, constrói a conexão com os seguintes parâmetros e valores padrão:

Variável	Padrão no código
<code>POSTGRES_HOST</code>	<code>localhost</code>
<code>POSTGRES_PORT</code>	<code>5432</code>
<code>POSTGRES_DB</code>	<code>usp_data_lake</code>
<code>POSTGRES_USER</code>	<code>usp</code>
<code>POSTGRES_PASSWORD</code>	<code>usp123</code>

O driver `psycopg` é importado somente quando uma conexão é solicitada. Se o pacote estiver ausente, `_load_psycopg` produz `RuntimeError` com indicação da dependência esperada.

5.3 Esquema relacional

`ensure_chunks_table` executa a criação idempotente da tabela `chunks` e de seu índice único. O esquema definido no código é:

Coluna	Tipo	Restrição ou padrão
<code>id</code>	<code>SERIAL</code>	chave primária
<code>documento_id</code>	<code>VARCHAR(255)</code>	não nulo
<code>chunk_id</code>	<code>INTEGER</code>	não nulo
<code>texto</code>	<code>TEXT</code>	não nulo
<code>título</code>	<code>TEXT</code>	opcional
<code>autor</code>	<code>TEXT</code>	opcional
<code>data_publicacao</code>	<code>TEXT</code>	opcional
<code>categoria</code>	<code>TEXT</code>	opcional
<code>url</code>	<code>TEXT</code>	opcional
<code>source_object</code>	<code>TEXT</code>	opcional
<code>created_at</code>	<code>TIMESTAMPTZ</code>	não nulo; padrão <code>NOW()</code>
<code>updated_at</code>	<code>TIMESTAMPTZ</code>	não nulo; padrão <code>NOW()</code>

O índice único `uq_chunks_documento_chunk` abrange (`documento_id`, `chunk_id`). Portanto, a identidade lógica de um fragmento é o par formado pelo documento e pela posição do chunk, enquanto `id` fornece a identidade numérica usada na integração com o Qdrant.

5.4 Upsert textual e transações

`INSERT_CHUNK_SQL` utiliza `INSERT ... ON CONFLICT (documento_id, chunk_id) DO UPDATE`. Em conflito, `texto`, `título`, `autor`, `data`, `categoria`, `URL` e objeto de origem são atualizados, e `updated_at` recebe `NOW()`. A instrução retorna o `id` e a expressão booleana (`created_at = updated_at`) `AS inserted`.

`insert_chunks` converte os campos obrigatórios para string ou inteiro por `_chunk_params`, executa uma instrução para cada registro e acrescenta ao dicionário resultante:

- `id`, com o identificador retornado;
- `postgres_id`, com o mesmo valor;
- `_inserted`, com o booleano retornado pela instrução SQL.

Quando a função cria a própria conexão, ela executa `commit` após todo o lote, `rollback` em caso de exceção e `close` no bloco final. Quando recebe uma conexão externa, não assume essas operações, deixando o controle transacional e o encerramento para o chamador.

5.5 Normalização de URL

`normalize_document_url` normaliza uma URL já fornecida sem realizar acesso à rede. A função:

- remove espaços nas extremidades;
- converte esquema e host para minúsculas;
- utiliza / quando o caminho está vazio;
- elimina o fragmento;
- remove parâmetros cujo nome começa com `utm_`;
- preserva e recompõe os demais parâmetros de consulta.

Essa normalização é empregada na detecção de colisões de identidade documental, não na obtenção ou validação remota da URL.

5.6 Resolução de colisões documentais

`resolve_document_id_collisions` procura, na tabela `chunks`, URLs já associadas aos `documento_id` recebidos. Para cada registro, compara a URL normalizada com o conjunto de URLs conhecidas para o mesmo identificador.

Quando o identificador já existe e a nova URL não pertence ao conjunto conhecido, a função deriva uma nova identidade. A entrada do hash é a URL normalizada ou, quando ela é vazia, o `source_object` ou o identificador original. O novo valor assume o formato `site_` seguido dos primeiros 16 caracteres hexadecimais de um SHA-256.

A função retorna os registros possivelmente remapeados e uma lista de colisões com identificador original, identificador resolvido e URL normalizada. O teste `test_reused_sequential_id_is_remapped` em `tests/test_corpus_identity.py`, verifica que chamadas equivalentes produzem o mesmo identificador e que ele segue o formato codificado.

5.7 Configuração do Qdrant

`src/qdrant_loader.py` obtém a URL de `QDRANT_URL`, com padrão `http://localhost:6333`, e utiliza `QDRANT_API_KEY` quando presente. O nome da coleção é obtido de `QDRANT_COLLECTION`, com padrão `usp_news_embeddings`.

Assim como o driver PostgreSQL, os tipos do `qdrant-client` são importados sob demanda. A ausência do pacote é convertida em `RuntimeError` por `_load_qdrant_types`.

5.8 Criação da coleção vetorial

`ensure_collection` verifica a existência da coleção por `collection_exists` quando esse método está disponível. Para clientes sem esse método, tenta `get_collection` e interpreta qualquer exceção como ausência.

Se a coleção não existir, ela é criada com dimensão igual ao comprimento do primeiro vetor recebido e distância COSINE. Se já existir, a função apenas retorna seu nome; não há, nesse caminho, uma comparação explícita entre a dimensão recebida e a configuração previamente armazenada.

5.9 Estrutura dos pontos vetoriais

Cada ponto criado por `upsert_embeddings` contém:

- `id`: valor de `postgres_id` ou `id` do registro;
- `vector`: sequência convertida para números de ponto flutuante;
- `payload`: `postgres_id`, `documento_id`, `chunk_id`, `titulo`, `url`, `categoria` e `source_object`.

O texto integral, o autor e a data de publicação não são incluídos no payload definido por `_payload`. Esses campos permanecem disponíveis no PostgreSQL e são recuperados posteriormente pelo identificador compartilhado.

5.10 Upsert vetorial em lotes

`upsert_embeddings` aceita vetores em argumento separado ou no campo `embedding` de cada registro. Uma lista vazia de registros retorna zero. Quando uma lista de embeddings é fornecida, sua quantidade precisa coincidir com a quantidade de registros.

O tamanho padrão do lote é 64. Ao atingir esse tamanho, o código chama `client.upsert` com `wait=True`, acumula a quantidade processada e inicia um novo lote. Os pontos restantes são enviados ao final pelo mesmo procedimento. O uso do mesmo ID numérico em novos `upserts` permite que o Qdrant trate o ponto como a mesma identidade, de acordo com a operação de `upsert` invocada.

5.11 Integridade referencial entre os repositórios

Não há chave estrangeira de banco de dados entre PostgreSQL e Qdrant, pois são sistemas distintos. A integridade da associação é implementada na aplicação por três decisões coordenadas:

1. o PostgreSQL produz e retorna o identificador numérico do chunk;
2. o carregador Qdrant usa esse número como ID do ponto e como `postgres_id` no payload;
3. o Retriever utiliza os IDs retornados pela busca vetorial para consultar a tabela chunks.

Essa associação depende da ordem do pipeline e da preservação dos IDs. O código não implementa uma transação distribuída que reverta automaticamente o PostgreSQL se um upsert Qdrant posterior falhar, nem uma restrição externa que obrigue as duas bases a conter exatamente o mesmo conjunto de IDs. A consistência entre os repositórios é, portanto, coordenada pelo fluxo da aplicação.

Capítulo 6 — Embeddings e recuperação híbrida

6.1 Representação vetorial com BGE-M3

`src/embedding.py` define `BGEM3Embedder`, uma abstração sobre `SentenceTransformer`. O modelo é obtido da variável `EMBEDDING_MODEL`; na ausência dela, o valor padrão é `BAAI/bge-m3`. O construtor também aceita modelo e dispositivo explicitamente.

O argumento `normalize_embeddings` é verdadeiro por padrão e é encaminhado ao método `encode` do `SentenceTransformer`. Consequentemente, os vetores produzidos pela configuração padrão são solicitados de forma normalizada. O código não fixa numericamente a dimensão: a propriedade `dimension` consulta `get_sentence_embedding_dimension` no modelo carregado e produz erro quando o modelo não a informa.

Quando `HF_HUB_OFFLINE` assume `1`, `true`, `yes` ou `on`, o construtor passa `local_files_only=True` ao `SentenceTransformer`. Essa condição é compatível com a configuração offline e o volume de cache definidos no `Compose` da raiz.

6.2 Codificação de textos

`encode_texts` materializa o iterável recebido como lista. Uma lista vazia retorna imediatamente sem chamar o modelo. Para entradas não vazias, a função utiliza lote padrão de 16, encaminha a opção de barra de progresso e solicita a normalização configurada no objeto.

Se o resultado possuir `tolist`, ele é convertido por esse método; caso contrário, cada vetor é convertido individualmente para lista. `embed_records` aplica essa operação ao campo textual configurável, cujo padrão é `texto`, verifica a correspondência entre a quantidade de registros e vetores e devolve cópias dos registros acrescidas do campo `embedding`.

A interface de linha de comando de `src/embedding.py` permite configurar arquivo de entrada, arquivo de saída, modelo, dispositivo e tamanho do lote. A dimensão exibida ao final é obtida dinamicamente da propriedade `dimension`.

6.3 Configuração da recuperação

`src/retriever.py` centraliza os parâmetros em `RetrievalConfig`. Os valores padrão codificados são:

Parâmetro	Padrão
recuperação híbrida	habilitada
recuperação lexical	habilitada
candidatos iniciais	20
fontes finais	5
chunks por fonte	1
peso vetorial	1,0
peso lexical	2,0
constante RRF	60,0
peso do título no BM25	3,0
peso da categoria	1,5
peso do conteúdo	1,0
peso do autor	0,5
k1 do BM25	1,5
b do BM25	0,75

`RetrievalConfig.from_env` permite substituir esses valores por variáveis iniciadas por `RAG_`. O código impõe limites inferiores a candidatos, fontes, chunks por fonte, constante RRF e k1, e restringe b ao intervalo de zero a um.

6.4 Tokenização lexical

`tokenize` aplica normalização Unicode NFKD, remove marcas combinantes e converte o texto para minúsculas. Uma expressão regular extrai sequências formadas por letras ASCII e algarismos. Assim, caracteres acentuados são representados por sua forma sem diacríticos, e pontuação não integra os tokens.

`tokenize_query` remove uma lista explícita de palavras funcionais e expressões de enquadramento, como artigos, preposições, “quais”, “notícias”, “Jornal” e “USP”. `extract_query_terms` elimina repetições preservando a ordem de primeira ocorrência.

O teste `test_tokenization_normalizes_accents_and_preserves_ia` confirma que “Inteligência, IA!” resulta nos tokens `inteligencia` e `ia`, e que os termos de enquadramento são retirados da consulta usada no teste.

6.5 Construção do índice BM25

`BM25Index` é construído em memória a partir de todos os chunks recuperados do PostgreSQL. Para cada documento, calcula frequências ponderadas nos campos título, categoria, texto e

autor. O comprimento do documento também é a soma ponderada das quantidades de tokens desses campos.

A frequência documental é contabilizada uma vez por termo presente em cada documento. O peso de frequência inversa utiliza a expressão codificada:

$$\text{idf} = \log(1 + (N - \text{df} + 0,5) / (\text{df} + 0,5))$$

Durante a busca, o score de cada documento é a soma das contribuições dos termos da consulta pela forma BM25 implementada com k_1 , b , frequência ponderada, comprimento do documento e comprimento médio. Somente documentos com score positivo integram o resultado. A ordenação usa score decrescente e, em empate, ID crescente.

6.6 Ciclo de vida do índice lexical

`Retriever._initialize_lexical_index` carrega todos os chunks com uma consulta PostgreSQL ordenada por `id` e constrói uma instância de `BM25Index`. A função verifica duas vezes se o índice já existe, antes e depois de adquirir `_lexical_lock`, impedindo sua construção repetida na mesma instância do `Retriever`.

O tempo dessa inicialização é registrado em `lexical_index_load_seconds`. O teste `test_index_built_once_and_reused` verifica que duas chamadas conservam o mesmo objeto e consultam os chunks apenas uma vez.

6.7 Busca vetorial

Na busca vetorial, `_embed_question` codifica uma lista contendo apenas a pergunta e desabilita a barra de progresso. A chamada é protegida por `_embedding_lock`, serializando o uso concorrente do encoder dentro da instância.

`_search_qdrant` oferece compatibilidade com duas interfaces do cliente. Quando existe `query_points`, chama esse método com `query`, `limite` e `payload` habilitado. Caso contrário, utiliza `search` com `query_vector`. Em ambos os caminhos, a coleção é obtida pela propriedade `collection`.

Para cada resultado, `_postgres_id` prioriza o campo `postgres_id` do `payload` e utiliza o ID do ponto como alternativa. Valores não conversíveis para inteiro produzem `ValueError`. O `Retriever` consulta então o PostgreSQL com `WHERE id = ANY(%s)` e constrói um dicionário indexado por ID.

A lista vetorial final é reconstruída percorrendo os hits na ordem recebida do Qdrant. Dessa forma, a ordem de relevância vetorial e o score de cada hit permanecem associados ao chunk correspondente, mesmo que a consulta SQL devolva as linhas em outra ordem. Hits sem registro PostgreSQL correspondente são omitidos.

6.8 Reciprocal Rank Fusion ponderado

`reciprocal_rank_fusion` reúne candidatos pelo ID numérico. Para cada candidato, registra `rank_vector`, `score_vector`, `rank_lexical` e `score_lexical` quando disponíveis. O score híbrido é calculado por:

```
score_hybrid = peso_vetorial / (k + rank_vector)
               + peso_lexical / (k + rank_lexical)
```

Uma parcela ausente contribui com zero. O valor híbrido é atribuído tanto a `score_hybrid` quanto ao campo público `score`. A ordenação utiliza, sucessivamente, score híbrido decrescente, rank vetorial, rank lexical e ID. O teste de fusão em `tests/test_hybrid_retriever.py` verifica candidatos presentes em uma ou nas duas listas, a aplicação dos pesos e a estabilidade do resultado.

6.9 Modos de recuperação e fallback

`search_with_metrics` admite `vector`, `lexical` e `hybrid`. Quando nenhum modo é informado, seleciona `hybrid` se a configuração híbrida estiver habilitada; caso contrário, seleciona `vector`. Modos diferentes desses três produzem `ValueError`.

O limite de candidatos intermediários é o maior valor entre `top_k` e `initial_candidates`. O modo vetorial executa embedding, Qdrant e PostgreSQL. O modo lexical utiliza apenas o índice BM25. O modo híbrido executa os dois caminhos e aplica RRF quando existem resultados lexicais.

Falhas durante a inicialização ou a busca lexical são capturadas e transformadas em lista lexical vazia. Nesse caso, o modo híbrido devolve os resultados vetoriais e registra `hybrid_fallback_vector`. O teste `test_hybrid_fallback_and_hybrid_disabled` cobre tanto esse fallback quanto o modo vetorial selecionado quando a recuperação híbrida está desabilitada.

6.10 Validação e deduplicação

`top_k` precisa ser um inteiro que não seja booleano e permanecer entre 1 e 20. A pergunta precisa ser uma string não vazia após remoção de espaços. Violações produzem `TypeError` ou `ValueError` antes da recuperação.

A deduplicação identifica a fonte pela URL quando disponível, pelo `documento_id` em seguida e, por último, pelo ID. A comparação usa diretamente o valor armazenado; não chama `normalize_document_url`. O algoritmo percorre o ranking já ordenado e mantém no máximo `max_chunks_per_source` resultados por chave.

O número final é o menor valor entre `top_k` e `final_sources`. Quando a deduplicação é desabilitada por argumento, a função apenas corta a lista nessa quantidade. O teste `test_deduplication_by_url_and_limit` confirma que dois chunks com a mesma URL são reduzidos a um quando o limite por fonte é 1.

6.11 Métricas do Retriever

`search_with_metrics` mede separadamente:

- carregamento inicial do índice lexical;
- geração do embedding;
- consulta ao Qdrant;
- consulta ao PostgreSQL;
- busca vetorial completa;
- busca lexical;
- fusão;
- deduplicação;
- tempo total do Retriever.

Também registra quantidades de candidatos vetoriais, lexicais, fundidos e deduplicados, quantidade final de fontes e modo efetivamente utilizado. `retriever_seconds` recebe o mesmo valor de `total_retriever_seconds`.

Capítulo 7 — Evidence Check e controle de suficiência

7.1 Posição no fluxo da RAG

O Evidence Check é implementado em `src/retriever.py` pelas estruturas `EvidenceConfig`, `EvidenceEvaluation`, `_source_evidence` e `evaluate_evidence`. Embora esteja localizado no módulo do Retriever, ele é invocado pela RAG API após `search_with_metrics` retornar os candidatos e antes de `prepare_context` e `generate_answer`.

Essa ordem torna a avaliação uma barreira determinística anterior ao modelo de linguagem. A função não chama o Ollama e não gera novos embeddings; ela utiliza a pergunta, o conteúdo textual das fontes e os sinais de ranking já anexados pelo mecanismo de recuperação.

7.2 Configuração padrão

`EvidenceConfig` define os seguintes valores padrão:

Parâmetro	Padrão
verificação habilitada	verdadeiro
fontes diretas mínimas	1
fontes relevantes mínimas	2
cobertura mínima dos termos	0,60
fontes com correspondência lexical mínimas	1
aceitar uma fonte forte	verdadeiro
cobertura mínima da fonte forte	0,80
exigir correspondência no título da fonte forte	falso
termos mínimos da fonte forte	2
posição lexical máxima da fonte forte	3
permitir parcial forte	verdadeiro
recusar quando todas forem fracas	verdadeiro

`EvidenceConfig.from_env` permite controlar esses parâmetros por variáveis `RAG_EVIDENCE_CHECK_ENABLED`, `RAG_MIN_DIRECT_SOURCES`, `RAG_MIN_TOTAL_RELEVANT_SOURCES`, `RAG_MIN_QUERY_TERM_COVERAGE`, `RAG_MIN_LEXICAL_MATCHED_SOURCES`, `RAG_ALLOW_SINGLE_STRONG_SOURCE`, `RAG_SINGLE_STRONG_SO`

RAG_SINGLE_STRONG_SOURCE_REQUIRE_TITLE_MATCH, RAG_SINGLE_STRONG_SOURCE_MIN_MATCHED_T, RAG_SINGLE_STRONG_SOURCE_MAX_LEXICAL_RANK, RAG_SINGLE_STRONG_SOURCE_ALLOW_STRONG_PAR e RAG_REJECT_ALL_WEAK.

O código restringe coberturas ao intervalo de zero a um, garante pelo menos duas correspondências para a fonte forte e impõe limites inferiores aos parâmetros de contagem e ranking.

7.3 Extração dos termos temáticos

A avaliação começa com `extract_query_terms`. A função reutiliza a tokenização normalizada da recuperação lexical, remove as palavras de enquadramento definidas em `QUERY_STOPWORDS` e elimina termos repetidos sem alterar a ordem.

Por exemplo, o teste `test_extracts_terms_accents_ia_and_removes_framing` comprova que a pergunta “Quais pesquisas da USP tratam de Inteligência Artificial e IA?” é reduzida a `inteligencia`, `artificial` e `ia`. Portanto, artigos, preposições e expressões genéricas não diminuem a cobertura temática calculada.

7.4 Sinais calculados por fonte

`_source_evidence` cria conjuntos de tokens separados para título e texto. A partir deles, calcula:

- termos cobertos pela união de título e texto;
- termos cobertos pelo título;
- termos cobertos pelo texto;
- proporção de cobertura total, do título e do texto;
- ocorrência da sequência completa dos termos normalizados;
- presença de sinal lexical positivo;
- presença de sinal vetorial positivo.

O sinal lexical exige `rank_lexical` não nulo e `score_lexical` maior que zero. O sinal vetorial exige `rank_vector` não nulo e `score_vector` maior que zero. O código considera forte o sinal lexical até a posição 10 e o sinal vetorial até a posição 5.

Cada fonte devolvida pela função é uma cópia enriquecida com classe, coberturas, frase exata, termos cobertos, posição híbrida e os indicadores `evidence_strong_partial` e `evidence_distributed_strong`.

7.5 Classes de evidência

As fontes são classificadas em três categorias:

1. **Direta:** cobertura praticamente integral, representada no código por valor mínimo de 0.999; rank lexical até 10; e frase exata, cobertura integral no título ou cobertura integral no texto.

2. **Parcial:** cobertura mínima de 0,60 acompanhada de sinal lexical ou vetorial forte; ou cobertura positiva com vetor forte quando não existe sinal lexical.
3. **Fraca:** qualquer fonte que não satisfaça as condições anteriores.

A classificação depende da presença literal dos termos normalizados no título ou texto. O score vetorial, isoladamente, pode classificar uma fonte como parcial quando há alguma cobertura textual, mas não a transforma automaticamente em evidência suficiente.

O teste `test_classifies_direct_partial_and_weak_deterministically` constrói uma fonte de cada classe e verifica que duas execuções produzem a mesma decisão e a mesma ordenação de classes.

7.6 Evidência parcial forte

Uma fonte parcial pode ser considerada defensável quando satisfaz o predicado `strong_partial`. Esse predicado exige, simultaneamente:

- classe parcial e permissão configurada;
- pelo menos dois termos temáticos;
- quantidade e cobertura mínimas configuradas;
- sinal lexical em posição não superior ao limite configurado;
- posição híbrida dentro do mesmo limite;
- frase exata, alta cobertura no título, alta cobertura no texto ou evidência distribuída forte;
- sinal vetorial forte, frase exata ou alta cobertura no título.

A condição `distributed_strong` atende perguntas com pelo menos quatro termos, dos quais pelo menos quatro estejam cobertos, com cobertura mínima configurada, vetor forte, correspondência lexical na primeira posição e primeira posição híbrida.

Os testes distinguem uma parcial forte em rank lexical 1 de uma correspondência semelhante em rank lexical 8, que permanece insuficiente. Também verificam evidência distribuída entre título e corpo e correspondência forte localizada somente no conteúdo.

7.7 Critério de aceitação por múltiplas fontes

Após classificar os candidatos, `evaluate_evidence` separa fontes diretas, parciais, parciais fortes, parciais fracas e fracas. O conjunto denominado `defensible` no código é formado pelas fontes diretas e parciais fortes.

A primeira condição de aceitação exige simultaneamente:

- quantidade mínima de fontes diretas;
- quantidade mínima de fontes defensáveis;
- cobertura conjunta mínima dos termos da pergunta;
- quantidade mínima de fontes relevantes com rank lexical presente.

A cobertura conjunta é calculada pela união dos termos cobertos por todas as fontes diretas e parciais. A diversidade é medida separadamente como a quantidade de URLs, documentos ou

IDs únicos entre as fontes relevantes e é registrada nas métricas, mas não integra diretamente a expressão booleana de aceitação.

7.8 Critério de fonte única forte

A segunda condição permite aceitar uma única fonte. A candidata escolhida é a primeira fonte direta ou, na ausência dela, a primeira parcial forte. Ela precisa satisfazer:

- permissão para fonte única;
- cobertura mínima configurada;
- quantidade mínima de termos cobertos;
- correspondência no título, apenas se essa exigência estiver habilitada;
- existência de rank lexical;
- rank lexical dentro do limite configurado.

Os testes comprovam aceitação de uma fonte com correspondência forte no título e de outra com correspondência forte no corpo. Também comprovam que uma consulta de termo único permanece recusada com a configuração padrão, pois a condição de fonte única exige pelo menos dois termos cobertos.

7.9 Regras de recusa

Antes de avaliar as duas condições de aceitação, a função aplica regras específicas:

- nenhuma fonte produz `no_results`;
- quando `reject_all_weak` está habilitado e todas as fontes são fracas, o motivo é `all_sources_weak`;
- quando o Evidence Check está desabilitado, todas as fontes são aceitas e o motivo é `evidence_check_disabled`.

Nos demais casos recusados, o motivo é escolhido sequencialmente entre `insufficient_term_coverage`, `insufficient_direct_sources`, `insufficient_relevant_sources` e `ambiguous_matches`, conforme a primeira condição não atendida no bloco de decisão.

O teste com uma pergunta sobre um “campeonato intergaláctico de xadrez quântico” verifica que uma correspondência superficial com outro campeonato continua recusada. Outro teste comprova que cinco fontes fracas são removidas das fontes públicas e fazem `ollama_skipped` assumir verdadeiro.

7.10 Fontes para contexto e para resposta pública

Quando a decisão é positiva, `context_sources` recebe somente o conjunto defensável: fontes diretas e parciais fortes. Quando a decisão é negativa, esse conjunto é vazio. Se a verificação estiver desabilitada, todos os candidatos originais são preservados como contexto.

`public_sources` recebe as fontes de contexto em decisões positivas. Nas recusas, recebe apenas as fontes diretas classificadas. Portanto, fontes fracas e parciais não defensáveis não

são encaminhadas ao Ollama e não são expostas como fundamento público de uma recusa.

7.11 Resultado e métricas

EvidenceEvaluation retorna a suficiência, a decisão, o motivo, os termos da pergunta, todas as fontes classificadas, as fontes de contexto, as fontes públicas e as métricas.

As métricas incluem tempo da avaliação, quantidades por classe, termos consultados e cobertos, cobertura conjunta, fontes com sinal lexical, fontes relevantes únicas, decisão, motivo e indicador de que o Ollama deve ser ignorado. Na RAG API, essas métricas são combinadas às métricas de recuperação antes da seleção do subconjunto permitido para a resposta HTTP.

7.12 Limite semântico do mecanismo

O Evidence Check implementado não constitui uma segunda inferência por modelo de linguagem. Sua decisão decorre de regras explícitas sobre tokens, cobertura e rankings. Desse modo, sua operação é reproduzível para entradas e configurações iguais, como verificado pelos testes de determinismo. Ao mesmo tempo, a própria implementação delimita sua análise à presença dos termos normalizados e aos sinais fornecidos pelo Retriever; não há, nessa etapa, uma avaliação semântica adicional além do ranking vetorial previamente calculado.

Capítulo 8 — Preparação de contexto e geração local com Ollama

8.1 Responsabilidade do serviço de linguagem

`src/llm_service.py` implementa a etapa de geração da RAG. O módulo possui duas responsabilidades principais: selecionar e limitar o contexto recuperado e realizar uma chamada HTTP à API local do Ollama.

A RAG API só utiliza essas operações depois que `evaluate_evidence` aceita as fontes. Portanto, o módulo de linguagem não escolhe quais candidatos são relevantes em primeiro nível; ele recebe o subconjunto considerado defensável pelo Evidence Check e aplica limites adicionais de duplicação, fonte e tamanho.

8.2 Configuração do Ollama

`OllamaConfig` representa a configuração imutável do serviço. Os padrões definidos em `src/llm_service.py` são:

Propriedade	Variável	Padrão do módulo
URL-base	<code>OLLAMA_BASE_URL</code>	<code>http://localhost:11434</code>
modelo	<code>OLLAMA_MODEL</code>	<code>gemma3</code>
timeout	<code>OLLAMA_TIMEOUT</code>	120 segundos
contexto máximo	<code>RAG_MAX_CONTEXT_CHARS</code>	6.000 caracteres
fontes máximas	<code>RAG_MAX_SOURCES</code>	3
temperatura	<code>OLLAMA_TEMPERATURE</code>	0,1
tokens de saída	<code>OLLAMA_NUM_PREDICT</code>	96
janela do modelo	<code>OLLAMA_NUM_CTX</code>	2.048
permanência em memória	<code>OLLAMA_KEEP_ALIVE</code>	10m

`get_ollama_config` remove a barra final da URL, elimina espaços externos do modelo e rejeita URL ou modelo vazios. Timeout, contexto máximo, fontes máximas, número máximo de tokens e janela de contexto passam por `_positive_number`, que exige inteiros estritamente positivos. A temperatura é convertida diretamente para `float`, e `keep_alive` vazio retorna ao padrão.

O Compose da raiz substitui parte dos padrões do módulo. Para a RAG API, ele define por padrão `OLLAMA_BASE_URL` como `http://host.docker.internal:11434`, `OLLAMA_MODEL` como `gemma3:4b` e `OLLAMA_TIMEOUT` como `300`. Também expõe as demais opções de geração e contexto com os mesmos valores padrão utilizados pelo módulo, exceto onde explicitamente indicado.

8.3 Reutilização do cliente HTTP

O cliente `httpx.Client` é armazenado na variável global `_client`. `get_http_client` aplica dupla verificação, antes e depois da aquisição de `_client_lock`, e cria o cliente somente quando ele ainda não existe. Requisições posteriores reutilizam a mesma instância.

`close_http_client` adquire o mesmo lock, fecha o cliente existente e redefine a referência como nula. Essa função é chamada no encerramento do lifespan da RAG API. A criação do cliente não ocorre durante a importação do módulo, pois depende da primeira chamada a `get_http_client`.

8.4 Filtragem de chunks

`prepare_context` percorre os chunks na ordem recebida. Entradas cujo campo texto não seja string ou esteja vazio após remoção de espaços são descartadas.

A identidade de duplicação é produzida por `_chunk_identity`. O texto é convertido por `casefold`, seus espaços são normalizados e o resultado é usado como chave. Embora a função receba o dicionário do chunk, ela o descarta explicitamente; logo, dois chunks com o mesmo texto normalizado são considerados duplicados mesmo que pertençam a documentos distintos.

O teste `test_deduplicates_text_groups_document_and_preserves_best_source` demonstra esse comportamento: quando um texto reaparece em outro documento, prevalece sua primeira ocorrência na ordem dos candidatos.

8.5 Agrupamento por documento

Após a deduplicação textual, os chunks são agrupados por `_document_identity`. A prioridade de identidade é `documento_id`, seguida de URL e ID. A ordem dos grupos corresponde à primeira ocorrência de cada documento na lista ranqueada.

Novos grupos deixam de ser criados quando `max_sources` é atingido. Entretanto, chunks adicionais de um documento já aceito podem ser incorporados ao mesmo grupo, desde que sejam textualmente distintos e caibam no orçamento de contexto.

8.6 Construção e truncamento do contexto

Cada grupo é apresentado como um bloco iniciado por:

[n] Título da fonte

Na ausência de título, o código utiliza `Sem título`. Os textos dos chunks do documento são concatenados em linhas subsequentes. O contexto enviado ao modelo contém título e texto; `prepare_context` não acrescenta URL, score, categoria ou identificadores ao bloco.

O orçamento considera separadores, prefixo e conteúdo. `_safe_truncate` preserva o texto integral quando ele cabe. Caso contrário, corta no limite e procura, em ordem de maior posição encontrada, um ponto seguido de espaço, uma quebra de linha ou um espaço. Esse limite semântico só é adotado quando ocorre após a metade do espaço disponível; caso contrário, o corte bruto é mantido.

Quando um chunk é truncado, o processamento interrompe a inclusão dos demais chunks daquele documento. Se não houver espaço para um novo prefixo, o processamento dos grupos termina. A função retorna a string de contexto e a lista dos chunks que efetivamente contribuíram com conteúdo.

O teste `test_respects_character_budget_without_discarding_first_source` verifica que o tamanho final não excede o orçamento e que a primeira fonte ranqueada permanece selecionada.

8.7 Prompt de sistema

`SYSTEM_PROMPT` instrui o modelo a responder perguntas sobre notícias do Jornal da USP somente com base no contexto fornecido. O texto proíbe conhecimento externo e a invenção de informações, títulos, URLs ou fontes. Também exige que insuficiência de contexto seja informada, que a resposta seja redigida em português brasileiro e que nomes, números, datas e listas sejam preservados.

As instruções ainda proíbem a repetição de URLs no texto da resposta e a exposição do raciocínio interno. Como as fontes estruturadas são adicionadas à resposta pela RAG API, a geração textual não é responsável por construir o contrato de fontes.

8.8 Requisição à API de chat

`generate_answer` constrói a URL pela concatenação de `base_url` com `/api/chat`. O corpo enviado contém:

```
{
  "model": "modelo configurado",
  "stream": false,
  "keep_alive": "valor configurado",
  "options": {
    "temperature": 0.1,
    "num_predict": 96,
    "num_ctx": 2048
  },
}
```

```

"messages": [
  {"role": "system", "content": "instruções do sistema"},
  {"role": "user", "content": "pergunta e contexto"}
]
}

```

Os valores numéricos do exemplo correspondem aos padrões do módulo. A mensagem do usuário usa os marcadores PERGUNTA: e CONTEXTO: e termina com a instrução para responder somente à pergunta. A requisição é não streaming e utiliza o timeout configurado.

O teste `test_payload_is_short_deterministic_and_uses_chat_endpoint` verifica o endpoint `/api/chat`, `stream=false`, `keep_alive`, as opções de geração e a extração do conteúdo da resposta.

8.9 Interpretação da resposta

Após uma resposta HTTP bem-sucedida, o código interpreta o JSON e procura `message.content`. O valor precisa ser uma string não vazia após remoção de espaços. Se a estrutura estiver ausente, possuir tipo incompatível, contiver JSON inválido ou resultar em texto vazio, a função lança `OllamaInvalidResponseError`.

Quando válido, o conteúdo é devolvido sem espaços externos. Não há pós-processamento adicional, validação factual por um segundo modelo ou streaming parcial na implementação examinada.

8.10 Taxonomia de falhas

O módulo define uma classe-base `LLMServiceError` e três especializações:

- `OllamaUnavailableError` para conexão, timeout e indisponibilidade HTTP;
- `OllamaModelNotFoundError` para modelo ausente;
- `OllamaInvalidResponseError` para resposta sem conteúdo utilizável.

Erros `httpx.ConnectError`, `httpx.TimeoutException` e demais `httpx.RequestError` são convertidos em indisponibilidade. HTTP 404 é interpretado como modelo ausente. Respostas 500 ou superiores são indisponibilidade. Para outros códigos a partir de 400, o corpo é inspecionado: a presença de “model” combinada a “not found” ou “não encontrado” produz erro de modelo; os demais casos produzem indisponibilidade.

A RAG API converte essas exceções em respostas HTTP controladas, conforme descrito no Capítulo 3. O teste `test_timeout_is_controlled` confirma que um `httpx.ReadTimeout` não é propagado diretamente, mas transformado em `OllamaUnavailableError`.

8.11 Dependência exclusivamente local no código analisado

Nos módulos `src`, nos dois arquivos de requisitos, nos testes e nos arquivos `Compose` analisados, não há referência a pacote `openai`, variável `OPENAI_API_KEY` ou endpoint `api.openai.com`. A implementação de geração importada pela RAG API é `src.llm_service`, e sua única chamada de modelo utiliza a URL configurada do Ollama.

Essa constatação descreve as dependências e referências presentes no repositório; ela não infere características de serviços externos além do endpoint efetivamente chamado pelo código.

Capítulo 9 — Containerização e infraestrutura de execução

9.1 Imagem comum das APIs

O Dockerfile da raiz define uma imagem comum para Generator API, Pipeline API e RAG API. A imagem-base é `python:3.12-slim`, e `PYTHONUNBUFFERED` recebe o valor 1. O diretório de trabalho é `/workspace`.

Durante a construção, são instalados `gcc`, `libxml2-dev` e `libxslt1-dev` sem pacotes recomendados. Em seguida, o cache de listas do APT é removido. O arquivo `requirements.txt` é copiado e instalado após atualização do `pip`; depois, `requirements-api.txt` é copiado e instalado separadamente. Por fim, o repositório é copiado para `/workspace`.

O Dockerfile não declara `CMD`, `ENTRYPOINT`, `EXPOSE` ou `HEALTHCHECK`. Os comandos, portas e verificações de integridade são definidos individualmente no Compose.

9.2 Dependências Python

`requirements.txt` reúne bibliotecas de coleta, transformação, persistência e embeddings:

- `requests`, `beautifulsoup4`, `pandas` e `lxml`;
- `minio` e `python-dotenv`;
- `markdownify` e `PyPDF2`;
- `sentence-transformers` e `torch`;
- `psycopg[binary]` e `qdrant-client`.

`requirements-api.txt` contém `fastapi`, `httpx` e `uvicorn[standard]`. Nenhum dos dois arquivos fixa versões por operadores de igualdade ou intervalo. Consequentemente, o conjunto de versões instalado é determinado pelo resolvidor no momento da construção, respeitadas as dependências transitivas disponíveis.

9.3 Composição principal

O `docker-compose.yml` da raiz atribui o nome `docker` ao projeto Compose e declara seis serviços:

Serviço	Imagem ou construção	Porta publicada	Responsabilidade no código
minio	minio/minio:latest	9000 e 9001	armazenamento Bronze e console
postgres	postgres:16	5432	persistência textual Gold
qdrant	qdrant/qdrant:latest	6333 e 6334	persistência vetorial Gold
generator-api	Dockerfile da raiz	8001 → 8000	coleta e geração Bronze
pipeline-api	Dockerfile da raiz	8002 → 8000	transformação e carga Gold
rag-api	Dockerfile da raiz	8003 → 8000	recuperação e geração de resposta

As tags dos serviços MinIO e Qdrant são latest, enquanto o PostgreSQL utiliza a tag principal 16. O arquivo não declara um serviço Ollama.

9.4 MinIO

O serviço MinIO executa `server /data --console-address ":9001"`. A API é publicada em 9000 e o console em 9001. O volume nomeado `minio_data` é montado em `/data`.

As variáveis `MINIO_ROOT_USER` e `MINIO_ROOT_PASSWORD` são declaradas diretamente no Compose. Generator API e Pipeline API recebem o endpoint interno `minio:9000`, as mesmas credenciais, o bucket `bronze` e `MINIO_SECURE=false`.

O serviço usa `restart: unless-stopped`. Não há bloco `healthcheck` declarado para o MinIO no arquivo examinado.

9.5 PostgreSQL

O serviço PostgreSQL utiliza `postgres:16`, publica a porta 5432 e monta `postgres_data` em `/var/lib/postgresql/data`. Banco, usuário e senha são declarados diretamente como `usp_data_lake`, `usp` e `usp123`.

O health check executa `pg_isready -U usp -d usp_data_lake`, com intervalo de 10 segundos, timeout de 5 segundos e cinco tentativas. Pipeline API e RAG API usam o nome de serviço `postgres` como host interno e recebem as mesmas informações de conexão.

9.6 Qdrant

O Qdrant publica 6333 e 6334 e monta `qdrant_data` em `/qdrant/storage`. Pipeline API e RAG API recebem `QDRANT_URL=http://qdrant:6333` e `QDRANT_COLLECTION=usp_news_embeddings`.

O serviço utiliza `restart: unless-stopped` e não possui `health check` declarado no Compose. O Compose não define explicitamente `QDRANT_API_KEY`. Entretanto, `src/qdrant_loader.py` carrega o arquivo `.env` da raiz antes de consultar a variável; como o repositório é montado em `/workspace`, uma chave definida nesse arquivo pode ser utilizada. Na ausência tanto de variável de ambiente quanto de valor no `.env`, o cliente recebe chave nula.

9.7 Execução das APIs

As três APIs utilizam contexto de construção `.` e o `Dockerfile` da raiz. Cada contêiner define `/workspace` como diretório de trabalho, monta o repositório nesse caminho e recebe `PYTHONPATH=/workspace`.

Os comandos são:

```
uvicorn src.api.generator_api:app --host 0.0.0.0 --port 8000
uvicorn src.api.pipeline_api:app --host 0.0.0.0 --port 8000
uvicorn src.api.rag_api:app --host 0.0.0.0 --port 8000
```

O código-fonte incorporado à imagem pelo `COPY` `.` `/workspace` é sobreposto, durante a execução via Compose, pelo `bind mount` do repositório. O sufixo `:cached` é declarado nos três `mounts`.

9.8 Dependências declaradas

`depends_on` estabelece as seguintes relações:

- Generator API depende do MinIO;
- Pipeline API depende de MinIO, PostgreSQL e Qdrant;
- RAG API depende de PostgreSQL e Qdrant.

O arquivo utiliza a forma de lista de `depends_on`, sem condições `service_healthy`. Assim, a configuração declara dependência de inicialização, mas não vincula explicitamente o início das APIs ao sucesso dos `health checks` dos serviços dependentes.

9.9 Health checks das APIs

Cada API possui `health check` executado no próprio contêiner com `urllib.request.urlopen` contra `http://localhost:8000/health`, usando `timeout` de 5 segundos. O intervalo é 30 segundos, o `timeout` do `health check` é 10 segundos e há cinco tentativas.

As funções `/health` retornam apenas um corpo de `status` e não consultam as dependências. Na RAG API, contudo, o `lifespan` é executado antes do atendimento HTTP e chama `Retrievever.initialize`, carregando o BGE-M3, construindo o índice BM25 a partir do PostgreSQL e obtendo cliente e coleção Qdrant. Assim, o endpoint não realiza essas operações, mas sua disponibilidade ocorre após a tentativa de inicialização de `startup`.

A RAG API acrescenta `start_period: 15m`. Esse período é coerente com a existência de inicialização do Retriever no lifespan, embora o código do Compose não registre a causa do valor escolhido. Generator API e Pipeline API não possuem `start_period`.

Todas as APIs e os três serviços de dados usam `restart: unless-stopped`.

9.10 Cache local do modelo de embeddings

Pipeline API e RAG API recebem:

```
HF_HOME=/models/huggingface
HUGGINGFACE_HUB_CACHE=/models/huggingface/hub
HF_HUB_OFFLINE=1
TRANSFORMERS_OFFLINE=1
```

O caminho `${HF_CACHE_HOST}` do hospedeiro é montado em `/models/huggingface` com modo somente leitura. Portanto, esses contêineres dependem de um cache preexistente e acessível para carregar o modelo quando o código respeita o modo offline. A variável `HF_CACHE_HOST` precisa ser resolvida pelo ambiente Compose ou por arquivo de variáveis.

A Generator API não recebe esse volume nem essas variáveis, pois seus módulos não instanciam `BGEM3Embedder`.

9.11 Comunicação com o Ollama no hospedeiro

A RAG API recebe como URL-base padrão `http://host.docker.internal:11434`. O bloco `extra_hosts` associa `host.docker.internal` a `host-gateway`. Essa configuração permite que o contêiner enderece o serviço disponível no hospedeiro pelo nome utilizado na variável.

Modelo, timeout, temperatura, limites de geração, permanência em memória e limites de contexto podem ser substituídos por interpolação de variáveis do ambiente. O Ollama não possui volume, health check, dependência ou política de reinício no Compose, pois não é gerenciado por esse arquivo.

9.12 Volumes e persistência

O Compose declara três volumes nomeados:

- `minio_data` para `/data`;
- `postgres_data` para `/var/lib/postgresql/data`;
- `qdrant_data` para `/qdrant/storage`.

Além desses volumes persistentes, os serviços de API utilizam bind mounts do código. Pipeline API e RAG API utilizam ainda o bind mount somente leitura do cache Hugging Face. Não há volume declarado para o Ollama.

9.13 Arquivo Compose alternativo

`docker/docker-compose.yml` declara os mesmos seis serviços, nomes de contêiner, portas, comandos e volumes nomeados. Como o arquivo está em um subdiretório, os contextos de construção e os mounts usam `..` para alcançar a raiz.

Comparado ao Compose principal, o arquivo alternativo não define:

- variáveis de cache Hugging Face;
- modo offline do Hugging Face e Transformers;
- mount de `${HF_CACHE_HOST}`;
- temperatura, `num_predict`, `num_ctx` e `keep_alive` do Ollama;
- limites de caracteres e fontes do contexto.

As configurações alternativas ausentes são então determinadas pelos padrões dos módulos Python ou pelo ambiente externo. Os arquivos são funcionalmente semelhantes quanto aos pontos de entrada, mas não idênticos quanto ao ambiente de execução.

9.14 Variáveis exemplificadas no repositório

`.env.example` reúne configurações para portas, MinIO, PostgreSQL, Qdrant, embeddings, Ollama, contexto, recuperação híbrida e Evidence Check. Ele utiliza endpoints locais para execução fora dos contêineres e apresenta `HF_CACHE_HOST` com um caminho ilustrativo contendo `SEU_USUARIO`.

As variáveis `GENERATOR_API_PORT`, `PIPELINE_API_PORT` e `RAG_API_PORT` aparecem no arquivo de exemplo, mas os mapeamentos de porta dos dois arquivos Compose estão escritos diretamente como 8001, 8002 e 8003; essas três variáveis não são interpoladas nos Compose examinados.

9.15 Delimitação de segurança da configuração

As credenciais padrão de MinIO e PostgreSQL estão registradas diretamente no código, no Compose e no `.env.example`. O código permite substituir os valores PostgreSQL e MinIO por variáveis de ambiente, mas o Compose principal fornece os valores declarados no próprio arquivo.

Não há configuração TLS para PostgreSQL ou Qdrant no Compose. Para o MinIO, `MINIO_SECURE` é explicitamente falso. Essas afirmações descrevem a configuração presente; não pressupõem exposição externa além dos mapeamentos de porta declarados.

Capítulo 10 — Testes automatizados e avaliação quantitativa

10.1 Organização da verificação

O diretório `tests` contém testes implementados com `unittest`. As classes utilizam `unittest.TestCase`, `mocks` de `unittest.mock` e, para as APIs, `fastapi.testclient.TestClient`. A inspeção dos arquivos identifica 64 métodos cujo nome começa com `test_`, distribuídos entre oito módulos:

Módulo	Objeto principal de verificação
<code>test_api_contracts.py</code>	contratos da Generator API e Pipeline API
<code>test_corpus_identity.py</code>	normalização de URL e colisões documentais
<code>test_evaluation.py</code>	esquema e cálculos do benchmark
<code>test_evidence.py</code>	classificação e decisão do Evidence Check
<code>test_hybrid_retriever.py</code>	tokenização, BM25, RRF e Retriever
<code>test_llm_service.py</code>	contexto e cliente Ollama
<code>test_pdf_flow.py</code>	geração, upload, extração e processamento de PDF
<code>test_rag_lifecycle.py</code>	ciclo de vida, contrato e concorrência da RAG API

Essa contagem descreve os métodos presentes nos arquivos, não o resultado de uma execução. Este relatório não infere aprovação da suíte apenas pela existência dos testes.

10.2 Testes de contratos HTTP

`tests/test_api_contracts.py` instancia clientes de teste para Generator API e Pipeline API. Os testes de health check comparam o corpo completo às estruturas definidas pelas APIs.

O teste do endpoint `/site` substitui a função real de scraping por um mock e verifica o status HTTP, o nome do serviço e a quantidade de artigos. O teste de `/processar-site` substitui `_run_pipeline`, evitando reproprocessamento Gold, e confirma a preservação do corpo retornado. Outro teste envia `limit=0` e espera HTTP 422, verificando a restrição Pydantic vigente.

Esses testes são de contrato e isolamento: não comprovam conectividade real com o Jornal da USP, MinIO, PostgreSQL ou Qdrant.

10.3 Testes do fluxo PDF

`tests/test_pdf_flow.py` cobre tanto a Generator API quanto a Pipeline API. Na geração, verifica rejeição de URL sem HTTP ou HTTPS, geração de PDF a partir de HTML simulado e chamada única ao upload. O arquivo produzido no teste é inspecionado pela assinatura %PDF- e removido ao final da própria rotina.

O upload Bronze é testado com diretório temporário. Um arquivo vazio precisa produzir `ValueError`; um conteúdo PDF não vazio, com cliente MinIO simulado, precisa usar o prefixo `raw/pdf/`.

Na transformação, os testes verificam PDF vazio, assinatura inválida, extração de texto, preservação da URL armazenada no metadado, identificador iniciado por `pdf_` e produção de chunks. A rota `/processar-pdf` é testada com persistência desabilitada e mocks, confirmando extensão `pdf`, prefixo `raw/pdf/` e ausência de chamadas PostgreSQL e Qdrant. O alias `/pdf` também é verificado no OpenAPI como depreciado.

10.4 Testes de identidade do corpus

`tests/test_corpus_identity.py` utiliza conexões PostgreSQL simuladas. A suíte verifica que fragmentos e parâmetros `utm_` são retirados da URL normalizada, que a combinação de mesmo identificador e mesma URL é preservada e que a reutilização do identificador para outra URL produz remapeamento determinístico no formato `site_` seguido por 16 caracteres hexadecimais.

Os testes não alteram uma tabela real; eles exercitam a regra de identidade com cursores mockados.

10.5 Testes do Retriever híbrido

`tests/test_hybrid_retriever.py` separa BM25, fusão e orquestração do Retriever.

Nos testes BM25, são verificados normalização de acentos, preservação da sigla IA, remoção de palavras de enquadramento, retorno vazio para consulta vazia ou sem correspondência, score positivo, determinismo e maior influência do título sob os pesos padrão.

O teste RRF constrói candidatos exclusivos e compartilhados entre rankings vetorial e lexical. Ele verifica pesos, posições ausentes, fórmula do score combinado, candidato vencedor e repetibilidade.

Os testes do Retriever verificam preservação de `source_object`, construção única do índice lexical, deduplicação por URL, execução lexical sem encoder nem Qdrant e fallback vetorial quando não há busca lexical disponível. Embedder, Qdrant e fábrica PostgreSQL são simulados, de modo que a finalidade é validar a lógica interna sem serviços externos.

10.6 Testes do Evidence Check

`tests/test_evidence.py` cobre respostas positivas e recusas. Entre os casos implementados estão:

- remoção de palavras funcionais sem perda de cobertura temática;
- evidência distribuída entre título e corpo;
- classes direta, parcial e fraca;
- fontes complementares;
- fonte única forte no título ou no texto;
- parcial forte e parcial com rank lexical insuficiente;
- consulta de termo único;
- correspondência superficial;
- candidato sem sinal lexical;
- conjunto integralmente fraco;
- verificação desabilitada.

Os testes também repetem a avaliação para comprovar igualdade de decisão em entradas idênticas. Uma pergunta deliberadamente externa ao conteúdo das fontes é utilizada para confirmar a recusa após a normalização dos termos.

10.7 Testes de contexto e Ollama

`tests/test_llm_service.py` verifica que textos duplicados são removidos mesmo entre documentos diferentes, que chunks de um mesmo documento são agrupados e que a melhor fonte permanece quando o orçamento é limitado.

O cliente HTTP é substituído por mock. O teste do payload verifica URL `/api/chat`, ausência de streaming, `keep_alive`, temperatura, `num_predict`, `num_ctx` e extração do conteúdo da resposta. Um `httpx.ReadTimeout` simulado precisa produzir `OllamaUnavailableError`.

Portanto, esses testes não exigem uma instância Ollama real e não avaliam a qualidade linguística de um modelo em execução.

10.8 Testes do ciclo de vida da RAG API

`tests/test_rag_lifecycle.py` define `FakeRetriever` com contadores de criação, inicialização e encerramento. Um teste gera o OpenAPI após a importação e confirma que nenhuma instância foi criada ou inicializada. Outro abre o lifespan por `TestClient`, executa duas perguntas e verifica uma única criação e uma única inicialização, reutilização entre requisições e encerramento ao sair do contexto.

O módulo também verifica:

- campos obrigatórios da resposta de `/pergunta`;
- preservação do score e dos metadados de fonte;
- compatibilidade com fontes sem os metadados novos;
- derivação determinística de `source_type`;

- campos opcionais no OpenAPI;
- logs de diagnóstico sem texto integral dos documentos;
- HTTP 503 controlado quando a inicialização falha;
- mensagem de ausência limitada ao corpus indexado;
- ausência de chamada ao Ollama diante de evidência fraca;
- exclusão de fontes fracas quando existe uma fonte forte;
- serialização do uso concorrente do encoder pelo lock.

Retriever, preparação de contexto e geração são substituídos conforme a finalidade de cada caso. Os testes avaliam a coordenação da API, não uma execução ponta a ponta com todos os serviços reais.

10.9 Estrutura do benchmark

O diretório `evaluation` implementa uma avaliação quantitativa separada da suíte unitária. Seus principais artefatos são:

- `questions.json`, com perguntas categorizadas;
- `ground_truth.json`, com fontes e respostas esperadas;
- `corpus_inventory.json`, consumido pelo gerador de relatório;
- `metrics.py`, com validação e métricas;
- `run_benchmark.py`, com execução contra a API;
- `report.py`, com produção do relatório de avaliação.

Os arquivos de perguntas e ground truth usam a versão `0.1.0`. O conjunto contém 20 casos: dez classificados como JSON, quatro como PDF e seis como fora do corpus. Os casos observados possuem `review_status=pending`, condição também verificada por `tests/test_evaluation.py`.

10.10 Validação do ground truth

`validate_ground_truth` exige uma lista não vazia e um conjunto explícito de campos por caso. IDs duplicados são rejeitados. O status de revisão deve ser `pending`, `approved` ou `rejected`. Quando `require_approved=True`, qualquer caso não aprovado impede a continuação.

Casos respondíveis precisam conter tipo, documento, título, URL e objeto de origem esperados, além de trecho e resposta de referência. `validate_benchmark` compara os conjuntos de IDs das perguntas e do ground truth e rejeita divergências em qualquer direção.

O teste correspondente confirma que o benchmark do repositório é carregável e que seus 20 casos estão pendentes. Outros testes verificam ausência de ground truth e bloqueio de casos ainda não aprovados para métricas finais.

10.11 Execução do benchmark

`evaluation/run_benchmark.py` envia requisições reais para `POST /pergunta`. O endpoint padrão é `http://localhost:8003/pergunta`, o `top_k` padrão é 5 e o timeout padrão é 420 segundos.

O executor é sequencial. Para cada caso, registra status HTTP, resposta, decisão de evidência, uso do Ollama, motivo da recusa, fontes sanitizadas, métricas retornadas e tempo de parede. O texto integral das fontes não integra o resultado sanitizado.

Os resultados são acrescentados em JSON Lines. `load_completed` permite identificar checkpoints concluídos e ignorá-los em nova execução. Um status diferente de 200 interrompe o benchmark após registrar o caso. Por padrão, todos os casos precisam estar aprovados; `--allow-pending` libera apenas execução preliminar.

Ao contrário dos testes unitários, esse executor não aplica mocks e depende de uma RAG API acessível no endereço configurado.

10.12 Métricas de classificação e recuperação

`evaluation/metrics.py` classifica um caso respondível como verdadeiro positivo somente quando a API aceita a evidência e uma fonte esperada é encontrada. Caso contrário, ele é falso negativo. Um caso não respondível aceito é falso positivo; um recusado é verdadeiro negativo.

A partir desses valores, o código calcula `accuracy`, `precision`, `recall`, `F1-score`, `specificity`, `balanced accuracy`, taxa de falsos positivos e taxa de falsos negativos. Divisões por zero retornam nulo em vez de lançar erro.

As métricas de recuperação incluem `Hit Rate` e `Recall` em 1, 3 e 5, além de `MRR`. Como o benchmark preliminar define uma relevância binária por caso, `ndcg_at_3` é explicitamente nulo e acompanhado de uma nota explicativa.

10.13 Latência e avaliação humana

As latências são agregadas para todos os resultados, por categoria de origem e segundo execução ou omissão do Ollama. Para cada fase, o código calcula quantidade, média, mediana, mínimo, máximo, desvio-padrão populacional e percentis 50, 90 e 95.

A avaliação humana utiliza cinco critérios: fidelidade, cobertura, ausência de informação não sustentada, fonte correta e clareza. Cada nota precisa ser um inteiro entre zero e dois. O código calcula a média por critério apenas para resultados que possuam avaliação humana explícita.

10.14 Proteção contra conclusões prematuras

`evaluation/report.py` distingue relatório preliminar de relatório final. Enquanto nem todos os casos estiverem aprovados, o relatório informa que as métricas não são resultados oficiais e não apresenta matriz de confusão, métricas de recuperação, latência ou gráficos como resultados calculados.

Mesmo com ground truth integralmente aprovado, a ausência de resultados é declarada. Somente a combinação de revisão aprovada e resultados permite calcular as métricas. O relatório gerado também inclui um aviso de que os resultados são válidos apenas para o conjunto documental local e não representam todo o portal.

10.15 Alcance da evidência de testes

O conjunto de testes demonstra que o repositório contém verificações automatizadas para contratos, regras algorítmicas, tratamento de falhas, lifecycle e avaliação. Contudo, a presença dessas rotinas não comprova seu resultado atual, pois nenhuma execução é pressuposta neste capítulo. Da mesma forma, os testes com mocks não substituem validações de conectividade, persistência real, disponibilidade do modelo ou desempenho no hardware de execução.

Capítulo 11 — Scripts operacionais e correspondência do README

11.1 Scripts auxiliares

O diretório `scripts` contém quatro programas relacionados ao inventário e à validação operacional:

Script	Finalidade codificada
<code>build_corpus_manifest.py</code>	inspecionar MinIO, PostgreSQL e Qdrant e produzir manifesto
<code>validate_existing_urls.py</code> <code>validate_rag_demo.py</code>	verificar URLs já registradas no manifesto executar um conjunto curto de perguntas contra a RAG API
<code>test_corpus_rag_e2e.py</code>	executar perguntas reais categorizadas contra a RAG API

Esses programas não são importados pelas três APIs nem executados pelos comandos do `Compose`. Eles constituem ferramentas separadas, acionadas por suas respectivas funções `main`.

11.2 Construção do manifesto do corpus

`scripts/build_corpus_manifest.py` consulta todos os objetos do bucket MinIO configurado. Para cada objeto, lê os bytes, classifica-o como PDF ou JSON pela extensão, reutiliza os decodificadores e o transformador de `src.transform` e calcula SHA-256, tamanho, caracteres, chunks e metadados.

O script chama `resolve_document_id_collisions` sobre todos os registros transformados. Em seguida, consulta a contagem de chunks por documento no PostgreSQL e percorre os pontos Qdrant com `scroll`, sem solicitar os vetores. As contagens são associadas aos documentos resolvidos.

O resultado contém escopo, bucket, documentos, colisões e resumo. A função `main` grava esse conteúdo em `data/recorte_manifest.json`. Portanto, embora o manifesto apenas

consulte os serviços e não altere seus dados, a execução do script cria ou substitui um arquivo local de relatório.

11.3 Validação das URLs existentes

`scripts/validate_existing_urls.py` lê exclusivamente as URLs do Jornal da USP presentes no manifesto. Ele não descobre links em páginas nem percorre URLs encontradas no conteúdo.

As requisições usam uma implementação `NoRedirect`, cujo método `redirect_request` retorna nulo. O corpo lido é limitado a 512.000 bytes. O script tenta extrair título HTML e URL canônica declarada, registra status, redirecionamento e erro e repete individualmente com timeout de 60 segundos os casos inicialmente sem status.

A concorrência é limitada a quatro trabalhadores por `ThreadPoolExecutor`. O resultado é ordenado por URL e gravado em `data/recorte_url_report.json`, com totais de respostas 2xx, redirecionamentos, erros HTTP e erros de conexão.

11.4 Validação curta de demonstração

`scripts/validate_rag_demo.py` define oito perguntas: três categorizadas como JSON, duas como PDF e três como externas. Cada pergunta é enviada sequencialmente para `http://localhost:8003/pergunta`, com `top_k=5` e timeout de 240 segundos.

Para JSON e PDF, o critério codificado exige HTTP 200, evidência suficiente, Ollama não ignorado e presença de fontes. Para perguntas externas, exige HTTP 200, evidência insuficiente, Ollama ignorado e ausência de fontes. Os resultados e tempos medidos pelo cliente são gravados em `data/rag_demo_report.json`.

O script avalia o contrato e a decisão binária; ele não compara automaticamente a resposta textual a um ground truth nem executa avaliação humana.

11.5 Validação ampliada do corpus

`scripts/test_corpus_rag_e2e.py` define 17 perguntas distribuídas entre JSON, PDF, consultas multidocumentais e perguntas fora do recorte. As chamadas usam a mesma RAG API, `top_k=5` e timeout de 360 segundos.

Para casos com título ou URL esperados, o script procura igualdade entre esses valores e as fontes retornadas. Para recusas, exige ausência de fontes e a expressão “não encontrei informações suficientes” na resposta. O campo correto combina essas condições com HTTP 200.

O script atribui internamente `evidence_sufficient` por `bool(sources)`, em vez de copiar diretamente o campo homônimo do payload. Assim, seu relatório interpreta presença de fontes como evidência suficiente para essa variável específica. O arquivo final é `data/rag_corpus_e2e_report.json`.

Os números de objetos, chunks, registros, vetores, URLs, dimensão e tempos apresentados no README são registros de validações operacionais. Alguns deles também aparecem em arquivos de inventário e relatório do repositório. Entretanto, esses valores não são constantes produzidas pela lógica das APIs e podem variar com o estado dos volumes e bancos. Sua confirmação atual exige executar os scripts de inventário ou consultar os serviços; a simples leitura do código não a fornece.

11.10 Contagem de testes

O README informa a existência de 64 testes automatizados, valor correspondente aos 64 métodos `test_` presentes no diretório `tests`. Essa contagem descreve a estrutura da suíte; a aprovação de todos os casos ainda depende da execução do comando `python -m unittest discover -s tests -v`.

11.11 Instalação e execução documentadas

Os comandos de instalação do README correspondem aos dois arquivos de requisitos consumidos também pelo Dockerfile. A instrução para definir `HF_CACHE_HOST` é necessária para resolver o bind mount declarado no Compose principal, e a indicação de cache somente leitura corresponde ao sufixo `:ro`.

Os comandos de inicialização usam `docker compose -f docker-compose.yml`, selecionando a composição principal. O exemplo de demonstração consulta os health checks e envia ao endpoint `/pergunta` os campos aceitos por `PerguntaRequest`.

O payload de Pipeline apresentado com os dois carregamentos desabilitados é válido segundo `ProcessRequest` e faz `_run_pipeline` executar apenas a transformação, sem chamar PostgreSQL ou Qdrant.

11.12 Limites de correspondência documental

O README está majoritariamente alinhado aos pontos de entrada e algoritmos atuais. As distinções identificáveis por inspeção são:

- os valores de inventário e desempenho são resultados registrados, não propriedades invariáveis do código;
- o Compose alternativo é mantido para os mesmos pontos de entrada, mas não contém todas as configurações do Compose principal;
- o modelo padrão interno de `src/llm_service.py` é `gemma3`, enquanto o Compose e o `.env.example` usam `gemma3:4b` por padrão operacional;
- o Compose não declara `QDRANT_API_KEY`, mas o carregamento de `.env` permite que a variável seja fornecida por esse arquivo.

Essas distinções não alteram o fluxo arquitetural implementado, mas são relevantes para separar contrato de código, configuração de implantação e observações de uma execução específica.

Capítulo 12 — Síntese técnica, limitações e conclusão

12.1 Síntese arquitetural

A implementação analisada organiza um fluxo completo entre aquisição de conteúdo, armazenamento de objetos, transformação textual, persistência estruturada, representação vetorial, recuperação híbrida, controle de evidência e geração de resposta.

As responsabilidades são distribuídas entre três aplicações FastAPI e módulos especializados no pacote `src`. A Generator API grava os artefatos na Bronze local e pode enviá-los opcionalmente ao MinIO; PostgreSQL mantém texto e metadados Gold; Qdrant mantém os vetores; e Ollama realiza a geração textual. As APIs são executadas separadamente, mas compartilham a mesma imagem Python e os mesmos módulos de domínio.

O vínculo entre as duas representações Gold é estabelecido pelo ID PostgreSQL. O pipeline persiste o chunk textual, recupera esse identificador, gera seu embedding e utiliza o mesmo número como ID e payload do ponto Qdrant. Na consulta, o processo é invertido: o resultado vetorial fornece o ID, e o Retriever obtém o texto correspondente no PostgreSQL.

12.2 Propriedades de modularidade

O código separa interfaces HTTP de operações de coleta, transformação, persistência e recuperação. Essa divisão permite testar grande parte da lógica com mocks, como demonstrado nos módulos de teste.

As dependências externas são obtidas por funções ou propriedades específicas. PostgreSQL e Qdrant são carregados sob demanda; o modelo BGE-M3 é encapsulado por `BGEM3Embedder`; o cliente Ollama é encapsulado em `src.llm_service`; e os clientes de armazenamento são configurados por ambiente.

A RAG API possui tratamento de ciclo de vida mais explícito que as demais APIs. Ela inicializa um Retriever por instância da aplicação, reutiliza modelo, cliente Qdrant e índice lexical, protege o encoder e encerra os recursos conhecidos ao final do lifespan.

12.3 Propriedades de rastreabilidade

Cada chunk transformado conserva `documento_id`, `chunk_id`, `source_object` e metadados documentais. O PostgreSQL acrescenta um identificador numérico, e o Qdrant preserva esse valor em dois locais: no ID do ponto e em `postgres_id` no payload.

A resposta da RAG pode expor identidade documental, chunk, identidade PostgreSQL, tipo e objeto de origem, além de título, URL, texto e score. As métricas distinguem as principais fases de recuperação e geração. Os logs de evidência registram rankings e metadados sem incluir o texto integral do documento.

Essas estruturas permitem rastrear uma fonte retornada até o registro textual e o objeto de origem, desde que os identificadores permaneçam consistentes entre os repositórios.

12.4 Propriedades de controle factual

O fluxo possui dois controles anteriores à geração. O Retriever limita e deduplica os candidatos após combinar busca vetorial e lexical. Em seguida, o Evidence Check classifica as fontes com regras explícitas e seleciona somente evidências diretas ou parciais fortes para o contexto.

Quando a evidência é insuficiente, a RAG API não chama `generate_answer`, retorna `ollama_skipped=true` e utiliza uma mensagem que restringe a ausência de resultados ao conjunto atualmente indexado. Quando há evidência, o prompt proíbe conhecimento externo e invenção de fatos ou fontes.

Esses mecanismos reduzem o conjunto de informações entregue ao modelo, mas o código não implementa uma verificação posterior da resposta produzida pelo Ollama. Após validar a presença de `message.content`, o texto é retornado ao cliente sem uma segunda comparação automática com as fontes.

12.5 Persistência e idempotência

O PostgreSQL utiliza chave lógica única em (`documento_id`, `chunk_id`) e `upsert`. A resolução de colisões evita que um identificador sequencial previamente associado a outra URL seja reutilizado sem remapeamento. O Qdrant utiliza `upsert` com o ID PostgreSQL.

Essas operações permitem reprocessar as mesmas identidades sem criar novas linhas ou pontos para o mesmo ID. Entretanto, PostgreSQL e Qdrant não participam de uma transação distribuída. Uma falha após o `commit` PostgreSQL e antes da conclusão Qdrant pode deixar os repositórios temporariamente divergentes, e o código não contém uma rotina automática de compensação.

12.6 Limitações da camada de processamento

O chunking usa posições de caracteres com tamanho e sobreposição fixos. A função não procura limites de sentenças ou tokens, de modo que um corte pode ocorrer no interior de uma unidade linguística.

Na Pipeline API, o modelo de embeddings é instanciado dentro de `_run_pipeline` a cada requisição que habilita o Qdrant. Diferentemente da RAG API, essa aplicação não possui `lifespans` ou `cache` explícito do `embedder`. Portanto, o código não reutiliza obrigatoriamente a instância do modelo entre chamadas da Pipeline API.

A transformação retorna todos os chunks como lista em memória, e a codificação recebe o conjunto de textos produzido. Não há streaming de documentos, chunks ou vetores entre as etapas da rota HTTP.

12.7 Limitações da recuperação

O índice BM25 é integralmente construído em memória com todos os registros da tabela chunks e permanece estático durante a vida do Retriever. O código não atualiza esse índice automaticamente após uma nova carga realizada enquanto a RAG API está em execução. Para refletir novos chunks lexicais, a instância precisaria reconstruir o índice por um novo ciclo de inicialização ou intervenção equivalente, que não está exposta como endpoint.

A busca lexical captura qualquer exceção e prossegue com fallback vetorial. Esse comportamento preserva disponibilidade do fluxo híbrido, mas não propaga ao cliente a causa da falha lexical. A métrica `retrieval_mode` informa `hybrid_fallback_vector`, permitindo identificar que a fusão não ocorreu.

A deduplicação usa diretamente URL, documento ou ID, sem normalizar a URL nessa etapa. URLs textualmente diferentes que identifiquem o mesmo recurso podem, portanto, constituir chaves distintas para a deduplicação.

12.8 Limitações do Evidence Check

O Evidence Check utiliza correspondência de tokens e sinais de ranking. Flexões, sinônimos ou relações conceituais que não compartilhem tokens normalizados dependem do ranking vetorial para aparecer como candidatos, mas ainda precisam satisfazer as regras de cobertura codificadas para serem aceitos.

Os limiares são configuráveis por ambiente, porém não são derivados automaticamente do corpus. A alteração dos parâmetros modifica o equilíbrio entre aceitação e recusa e exige avaliação externa para determinar seu efeito.

A métrica de fontes únicas é calculada, mas não integra diretamente a condição de aceitação. A condição principal usa quantidades de fontes defensáveis e cobertura; a condição alternativa permite uma única fonte forte sob os requisitos definidos.

12.9 Limitações da geração

A chamada ao Ollama é síncrona e não utiliza streaming. O endpoint `/pergunta` também é uma função síncrona. O tempo da requisição inclui recuperação, Evidence Check, montagem do contexto e conclusão integral da geração.

O contexto contém título e texto, mas não inclui a URL ou o score. As URLs e demais metadados são adicionados posteriormente pela API a partir dos chunks selecionados. O modelo, portanto, não é solicitado a reproduzir nem validar esses campos.

O cliente HTTP é compartilhado dentro do processo, mas cada processo Python manteria sua própria variável global. Analogamente, a garantia de uma instância do Retriever é válida por instância da aplicação; o código não implementa coordenação entre múltiplos processos Uvicorn.

12.10 Limitações dos health checks

Os endpoints `/health` das três APIs retornam respostas constantes e não consultam MinIO, PostgreSQL, Qdrant, cache de embeddings ou Ollama. Na RAG API, essa característica do endpoint deve ser distinguida do startup: antes de atender requisições, o lifespan tenta inicializar o Retriever, carregando o BGE-M3, o índice BM25 e o cliente Qdrant. Uma resposta 200 comprova a disponibilidade do processo HTTP após essa tentativa, mas não a saúde individual de cada dependência, pois o endpoint não as consulta.

No Compose, PostgreSQL e as APIs possuem health checks. MinIO e Qdrant não possuem verificações próprias declaradas, e `depends_on` não utiliza condições de saúde. A RAG API trata falha de inicialização do Retriever e responde 503 em `/pergunta`, mas seu `/health` continua independente desse estado.

12.11 Limitações de configuração e implantação

As dependências Python não possuem versões fixadas. MinIO e Qdrant utilizam imagens com tag `latest`. Assim, novas construções podem resolver versões diferentes das utilizadas anteriormente.

Credenciais padrão aparecem no código e no Compose, e as portas dos serviços são publicadas no hospedeiro. As APIs não definem autenticação ou autorização em seus módulos FastAPI, e o Compose não declara proxy TLS. Essas características correspondem a uma configuração direta de execução local; o código analisado não acrescenta controles próprios para uma exposição em rede não confiável.

O cache BGE-M3 é montado como somente leitura e o modo offline é habilitado no Compose principal. A inicialização depende, portanto, de o caminho indicado por `HF_CACHE_HOST` conter os arquivos necessários. O Compose alternativo não estabelece essa mesma configuração.

12.12 Limitações de validação

A suíte automatizada utiliza mocks para isolar a maior parte das dependências. Essa estratégia verifica contratos e algoritmos, mas não substitui testes de integração com serviços reais.

O benchmark dispõe de executor real, ground truth, métricas e relatório, porém os casos atuais estão marcados como pendentes. O próprio código impede tratá-los como avaliação final sem

aprovação humana. Consequentemente, este relatório não apresenta métricas quantitativas de qualidade ou desempenho como resultado vigente.

Os valores empíricos registrados no README ou em relatórios representam estados de execução e não garantias do programa. Contagens atuais, latência e disponibilidade precisam ser verificadas no ambiente correspondente.

12.13 Conclusão

O repositório implementa uma cadeia tecnicamente integrada entre coleta, Data Lake Bronze, transformação, persistência Gold textual e vetorial, recuperação híbrida e geração local. A combinação de BGE-M3, Qdrant, BM25 e RRF produz candidatos associados aos registros PostgreSQL; o Evidence Check controla a passagem desses candidatos ao Ollama; e a FastAPI devolve resposta, fontes e métricas em contrato tipado.

A arquitetura codificada é modular e observável por identificadores, scores, rankings e tempos de fase. Seus limites também são explícitos: corpus dependente do estado externo, índice lexical em memória, ausência de transação distribuída, health checks superficiais, geração sem verificação posterior e configurações voltadas à execução local.

Todas as conclusões deste relatório derivam dos arquivos de implementação, configuração, testes e scripts presentes no repositório. Nenhuma conclusão operacional — como disponibilidade dos serviços, quantidade atual de registros, aprovação dos testes ou qualidade medida das respostas — é assumida sem execução correspondente.

Apêndice A — Diagramas do sistema

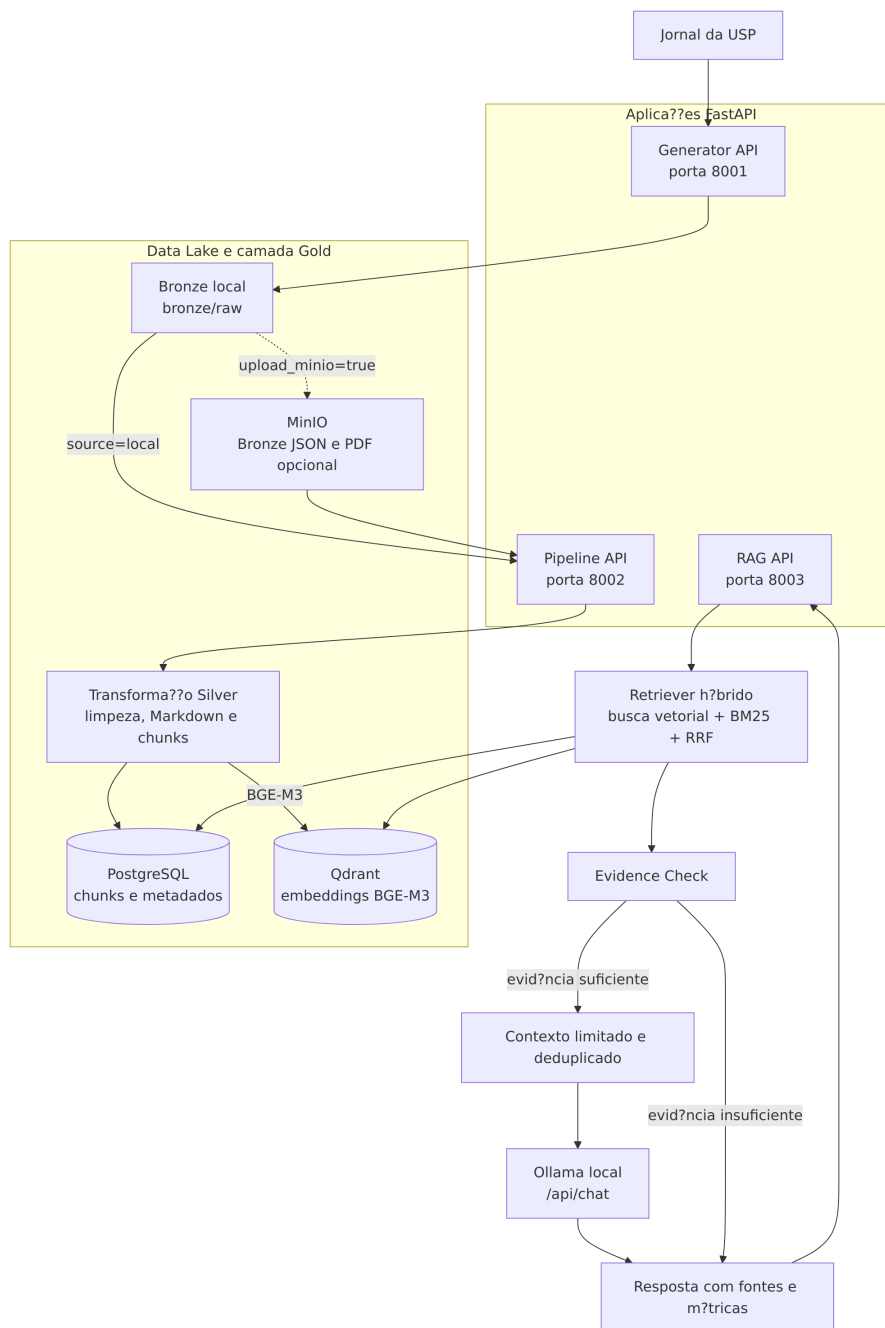


Figura 1: Arquitetura geral da plataforma

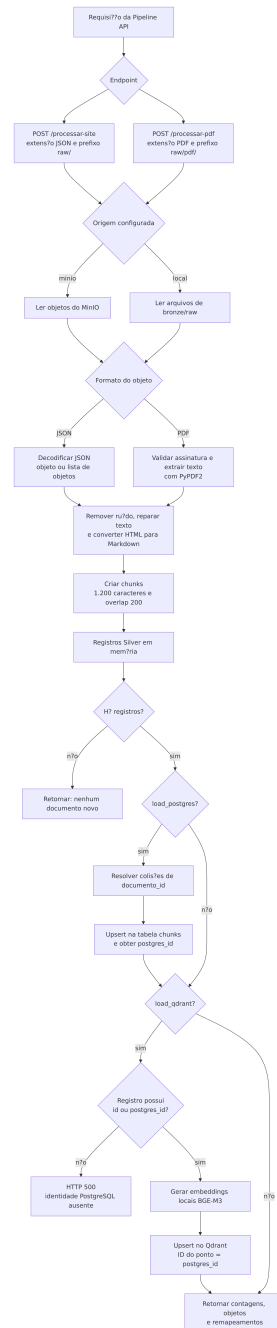


Figura 2: Pipeline de dados Bronze, Silver e Gold

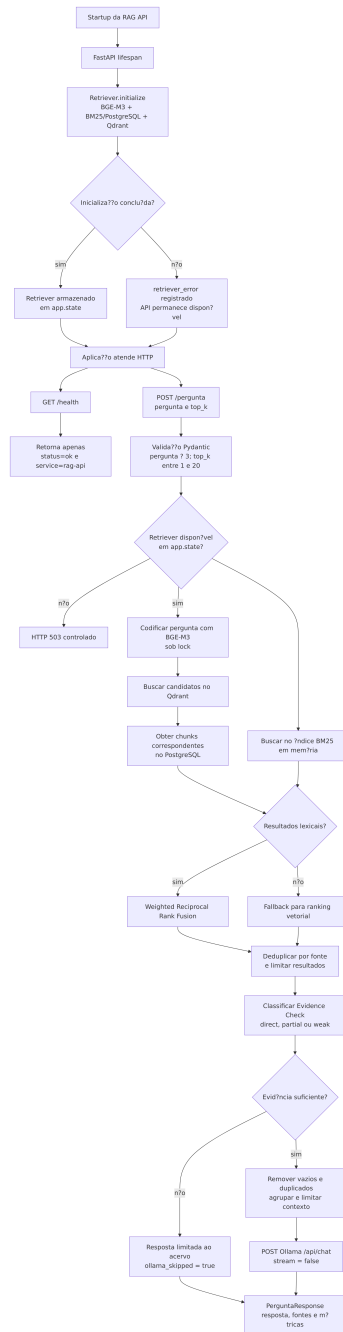


Figura 3: Fluxo interno da RAG

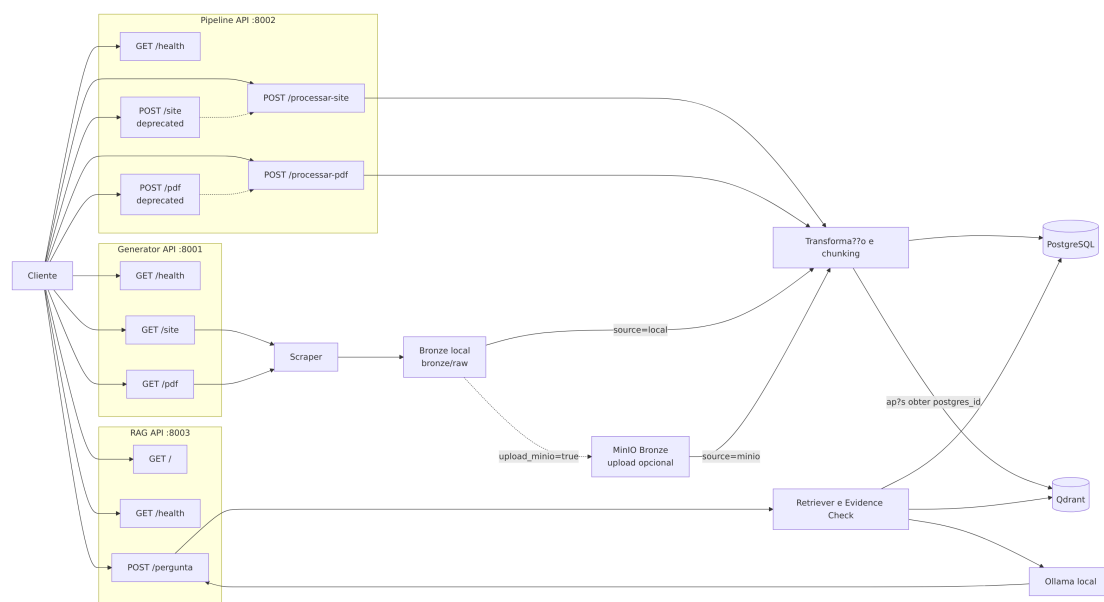


Figura 4: Fluxo das APIs

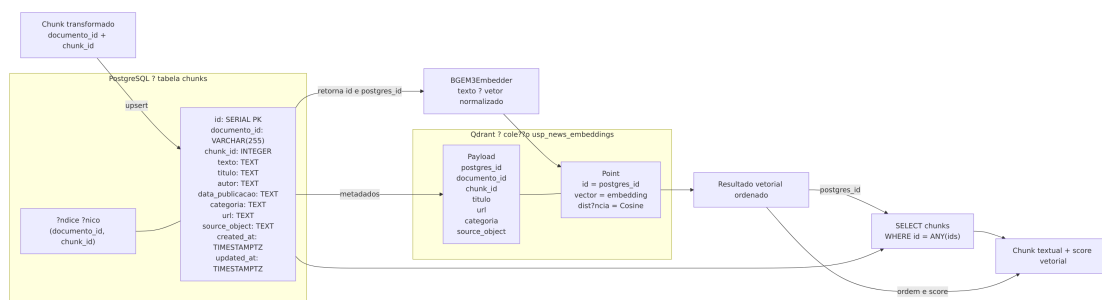


Figura 5: Persistência e identidade dos dados

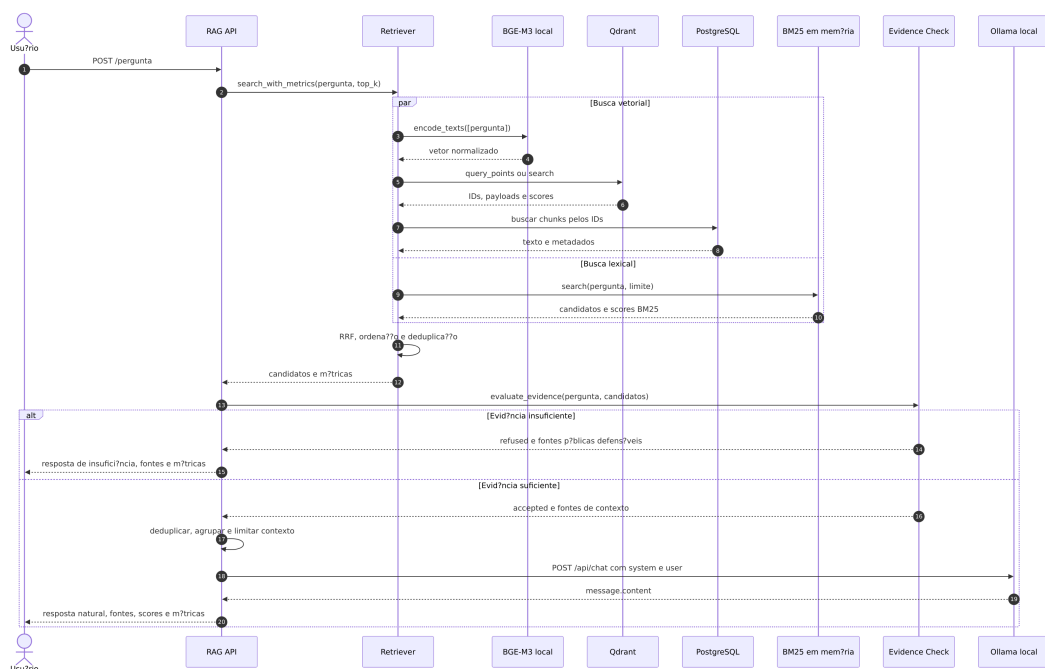


Figura 6: Fluxo conversacional do chatbot